

Video Processing

Lectures by Shai Avidan
Collected by Yaniv Korobkin

Spring 2025

Contents

1	Motion Representation	3
1.1	Motivation	3
1.2	Linear Transformation	3
1.3	Affine Transformation	4
1.4	Homography	5
1.5	Projection Matrix	6
2	Optical Flow	8
2.1	Introduction	8
2.2	Lucas Kanade	9
2.3	Transformations for optical flow	10
2.4	Lucas Kanade - private case	10
2.5	Harris Corner Detector	13
2.6	Applications	15
2.7	Horn-Shunck	15
3	Applications of Motion Estimation	19
3.1	Super Resolution	19
3.2	Hyperlapse	22
4	Kalman and Particle Filters	28
4.1	Tracking	28
4.2	Kalman Filter	30
4.3	Particle Filter	35
5	Texture synthesis	41
6	Layer image representation	45
6.1	Layer image representation	45
7	Video Magnification	50
8	Background Substruction	52

9	Video colorization	57
9.1	Dijkstra	61
9.2	Danielsson (Fast Sweeping)	63
9.3	Colorization in video	64
10	Video Matting	67
10.1	Trimap Estimation:	68
10.2	Trimap Refinement - Opacity map	71
10.3	Matting	72
10.4	KDE - Kernel Density Estimation	73
11	Re-targeting and Patch Match	75
11.1	Introduction	75
11.2	Re-targeting - Formalization	75
11.3	Patch Match	83
11.4	Generalization in Video	86
11.5	Inpainting	87
11.6	Re-shuffling	88
11.7	Additional Constraints	88

1 Motion Representation

1.1 Motivation

One of the main goals in video processing is estimating motion within an image. This requires a suitable mathematical representation, which can be obtained using linear algebra—either through motion vectors or transformation matrices. In today's lecture, we will examine 2D and 3D transformations. Transformations in 3D describe the motion of points in the world, while transformations in 2D describe how that motion is perceived by the camera.

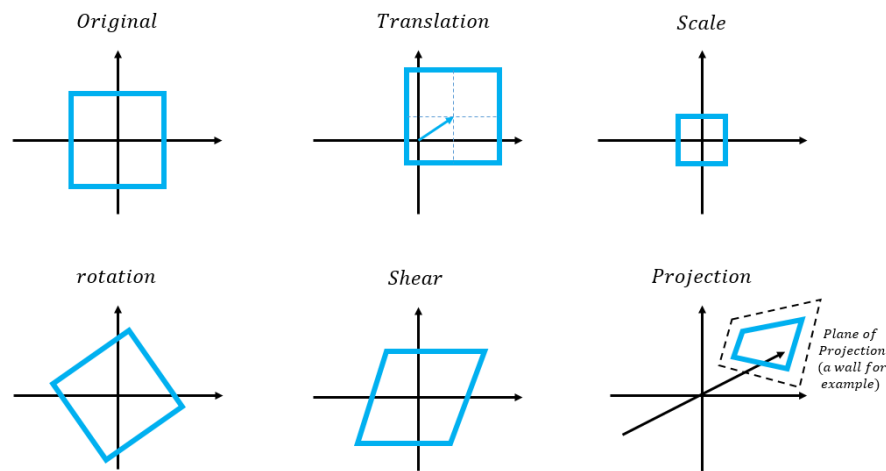


Figure 1: Examples of 2D and 3D transformations

1.2 Linear Transformation

We will define a point $p = \begin{pmatrix} x \\ y \end{pmatrix}$ and look at the transformation matrix:

$$p' = Ap$$

Where p' is in dimensions 2×1 and A is in dimensions 2×2 . Matrix A represents linear transformation and is able to do the following operations: rotation, scale, and shear. However, it cannot do translation because no matter what operation we do, applying any matrix on $(0,0)$ will always result in $(0,0)$.

Suppose now we want to estimate matrix A given matching points p and p'. We develop the following equations:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

$$\begin{pmatrix} x & y & 0 & 0 \\ 0 & 0 & x & y \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix} \begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix} = \begin{pmatrix} x' \\ y' \\ \vdots \end{pmatrix}$$

Or:

$$\tilde{A}\tilde{x} = b$$

Notice that each matching pair gives us two equations. Since we have 4 unknowns, we will need at least 2 matching pairs. If we get exactly two pairs, then we can recover $\tilde{x}^T = (a \ b \ c \ d)^T$ By inverting a 4×4 matrix. If we get more pairs, then we have more equations than unknowns. Usually it gives us a better global result, and to solve this we can use, for example, Least Squares (denoted later as LS).

1.3 Affine Transformation

The Affine transformation is a linear transformation with translation:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = (A \ t)_{[2 \times 3]} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Notice that now we add a third coordinate with value "1" to all the points. This makes it possible to move the origin point $(x, y) = (0, 0)$ to any (x', y') . If we want to be consistent with dimensions, we can do:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} A & t \\ 0_{[1 \times 2]} & 1 \end{pmatrix}_{[3 \times 3]} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Estimating A and t given a couple of matching points is done as before:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b & c \\ d & e & f \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} x & y & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x & y & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{pmatrix} \begin{pmatrix} a \\ b \\ c \\ d \\ e \\ f \end{pmatrix} = \begin{pmatrix} x' \\ y' \\ \vdots \end{pmatrix}$$

$$\tilde{A}\tilde{x} = b$$

Notice now we need at least 3 matching pairs instead of 2.

1.4 Homography

Homography is a transformation of the form:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} \cong H \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Where $\cong$ means up to scale. This makes the input and output to have the same dimension.

$$\begin{pmatrix} x'/w' \\ y'/w' \\ 1 \end{pmatrix} \cong \begin{pmatrix} x' \\ y' \\ w' \end{pmatrix} = Hp$$

This representation is called Homogenic Coordinates.

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

For simplicity, we write this as follows:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} - & H_1 & - \\ - & H_2 & - \\ - & H_3 & - \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Where H_i is row i of matrix H . The way we deal with up to scale is by dividing the last element:

$$\frac{x'}{1} = \frac{H_1 p}{H_3 p}$$

$$\frac{y'}{1} = \frac{H_2 p}{H_3 p}$$

Rearranging the equations gives:

$$p^T H_1^T = x' p^T H_3^T \implies (x \ y \ 1) H_1^T - x' (x \ y \ 1) H_3^T = 0$$

$$p^T H_2^T = y' p^T H_3^T \implies (x \ y \ 1) H_2^T - y' (x \ y \ 1) H_3^T = 0$$

$$\begin{pmatrix} x & y & 1 & 0 & 0 & 0 & -x'x & -x'y & -x' \\ 0 & 0 & 0 & x & y & 1 & -y'x & -y'y & -y' \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{pmatrix} \begin{pmatrix} H_1^T \\ H_2^T \\ H_3^T \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ \vdots \end{pmatrix}$$

$$Ax = 0$$

A has dimensions of $2N \times 9$. By having 4 matching points, we will get 8×9 , which is still not enough. As a result, in the last row we can choose a vector of ones as a constraint to normalize the values of H . This usually provides a stable solution and avoids large numerical values.

When solving a tracking problem, the more parameters the model has, the harder it becomes to estimate them reliably, since noise can significantly affect the solution. If the object is small and far away, a linear transformation may be sufficient. If it is closer, an affine transformation might be more appropriate, and if the object is large and close to the camera, a homography may be required.

1.5 Projection Matrix

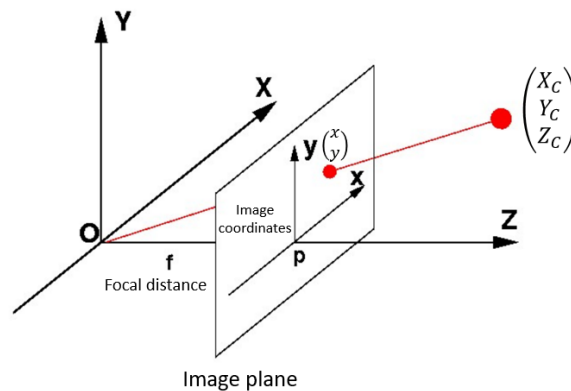
In the following section, we will show briefly how a point in the 3D world is projected onto a pixel in the image. The full pipeline will be:

$$\underbrace{\begin{pmatrix} u \\ v \\ 1 \end{pmatrix}}_{\text{pixel in the image}} = \underbrace{\begin{pmatrix} x \\ y \\ 1 \end{pmatrix}}_{\text{Camera coordinates (2D)}} \leftarrow \underbrace{\begin{pmatrix} X_C \\ Y_C \\ Z_C \\ 1 \end{pmatrix}}_{\text{Camera coordinates (3D)}} \leftarrow \underbrace{\begin{pmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{pmatrix}}_{\text{World coordinates (3D)}}$$

The main assumption is that we consider a pinhole camera model (without any lens). Suppose a point in 3D is described in the world coordinate system W. Our goal is to express this point in the camera coordinate system (denoted by C). If the camera is rotated by a rotation matrix R and translated by a translation vector T w.r.t. the coordinate system W, then we can describe this transformation mathematically as follows:

$$\begin{pmatrix} X_C \\ Y_C \\ Z_C \\ 1 \end{pmatrix} = \begin{pmatrix} R_{3 \times 3} & t_{3 \times 1} \\ 0_{1 \times 3} & 1 \end{pmatrix} \begin{pmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{pmatrix}$$

Next, we examine the geometry of the world and the image:



From triangle similarity rule:

$$\frac{x}{f} = \frac{X}{Z} \implies x = X \frac{f}{Z}$$

$$\frac{y}{f} = \frac{Y}{Z} \implies y = Y \frac{f}{Z}$$

Where f is a known camera parameter. If we work with Homogeneous coordinates:

$$\begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} X \\ Y \\ Z/f \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1/f & 0 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}$$

Finally, the transformation to pixel coordinates will be:

$$u = u_0 + k_u x$$

$$v = v_0 + k_v y$$

Where k_u, k_v are known camera parameters which transform from cm to pixels. This can be written as:

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} k_u & 0 & u_0 \\ 0 & k_v & v_0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

To conclude:

$$\begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \approx \underbrace{\begin{pmatrix} k_u & 0 & u_0 \\ 0 & k_v & v_0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1/f & 0 \end{pmatrix} \begin{pmatrix} R_{3 \times 3} & t_{3 \times 1} \\ 0_{1 \times 3} & 1 \end{pmatrix}}_P \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}$$

Where P is the projection matrix.

2 Optical Flow

2.1 Introduction

Optical flow is the basis for motion estimation. By definition, it is the projection of the motion in 3D onto a 2D image.

Input: The inputs are 2 images. The first image I_1 taken at some time t , and the 2nd image I_2 taken later (at some time $t + \Delta$).

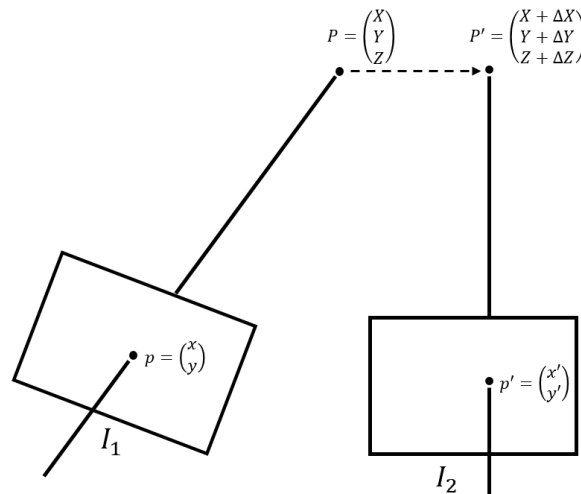
Output: For every pixel (x, y) we will find a motion vector (u, v) s.t. it's new location will be:

$$I_1(x + u, y + v) = I_2(x', y')$$

or:

$$x' = x + u$$

$$y' = y + v$$



Optical Flow asks for every pixel in the first image where it was transferred in the 2nd image. There are several problems with how we define the problem:

1. Not all points are visible in both images - some points entered the 2nd image for the first time, or some points from the first image are not inside the frame at the second image.
2. Some points are hidden behind other objects.
3. The values of (u, v) may be fractional.
4. We assume that the intensity and color of corresponding pixels in both images remain the same. This assumption is known as the **constant brightness constraint**. However, this assumption does not always hold. For example, the

observed brightness may change due to light reflections when viewed from different angles. Another example is a change in the intensity of the light source between frames. In addition, if the object is reflective, different viewing angles may produce different reflections of the environment on its surface.

2.2 Lucas Kanade

Let us assume for now that these assumptions hold, and that we want to solve the correspondence problem. One simple approach is to search, for every pixel in the first image, for a pixel in the second image with the same intensity. However, this solution is computationally very expensive.

The solution we'll offer uses the following assumption:

$$I_2(x, y) \approx I_1(x, y) + uI_x(x, y) + vI_y(x, y)$$

This is the first-order Taylor approximation. We assume that the two images were taken at a very short time interval. Thus, we can approximate the second image using the first image and its derivatives with respect to the x-axis and y-axis.

Instead of working with $I_1(x + u, y + v)$, we now have a direct relation between $I_1(x, y)$ and $I_2(x, y)$ and our only unknowns are u and v . We can now add constraints on u and v with the intensity measurements of the two images.

The problem is that we have 2 unknowns and only one equation. The solution will be by observing a local environment and get a set of equations:

$$\begin{pmatrix} I_x & I_y \\ \vdots & \vdots \\ I_x & I_y \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} I_t \\ \vdots \\ I_t \end{pmatrix}$$

Every row is related to a pixel in the environment (a window) and $I_t = I_2 - I_1$. We got an equation of the form:

$$Ax = b$$

Where our unknowns here are u and v . This is solved by Least Squares. We want to minimize:

$$\begin{aligned} E &= \|Ax - b\|^2 \\ E &= (Ax - b)^T (Ax - b) \\ \frac{\partial E}{\partial x} &= 2A^T(Ax - b) = 0 \\ x &= (A^T A)^{-1} A^T b \end{aligned}$$

If we take a window of size $n \times n$, A will have the dimensions of $n^2 \times 2$. This means that $A^T A$ will have dimensions of 2×2 . We got that to solve the problem, we need to invert a matrix of 2×2 and do simple multiplications of matrices and vectors.

This approach will not work if we want to use a very large window (or even the entire image). Some applications that require this include tracking and video stabilization.

2.3 Transformations for optical flow

We are given two images I_1, I_2 and define a point by $X = \begin{pmatrix} x \\ y \end{pmatrix}$. We will work with the following two transformations:

Translation transformation:

$$W(X; P) = \begin{pmatrix} x + p_1 \\ y + p_2 \end{pmatrix}$$

Affine transformation:

$$W(X; P) = \begin{pmatrix} p_1 x + p_2 y + p_3 \\ p_4 x + p_5 y + p_6 \end{pmatrix} = \begin{pmatrix} p_1 & p_2 & p_3 \\ p_4 & p_5 & p_6 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Where p_i are the parameters of the transformation.

2.4 Lucas Kanade - private case

The goal of LK is to minimize:

$$P^* = \arg \min_P \sum_X [I_2(W(X; P)) - I_1(X)]^2$$

This method uses the constant brightness constraint. In this case, we aim to estimate the global motion of the camera, which is useful for applications such as video stabilization. The difficulty is that the parameters appear inside the warp function. As a result, we must search for a solution in a six-dimensional parameter space, which is computationally expensive. However, if we take the parameters out from $W(X; P)$ it will be a lot easier to calculate them (for example with LS). We will solve this using the incremental algorithm.

Incremental Algorithm

We will iteratively try to solve the following:

$$\Delta P^* = \arg \min_{\Delta P} \sum_X [I_2(W(X; \Delta P)) - I_1(X)]^2$$

$$P = P + \Delta P^*$$

We don't solve right away for all the parameters. We start with some initial parameters (for example the identity matrix), and each time we estimate ΔP . After every iteration, we get closer and closer to the solution. An example on an initial P could be:

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Now we will use Taylor series approximation:

$$I_2(W(X; P + \Delta P)) \approx I_2(W(X; P)) + \nabla I_2 \frac{\partial W}{\partial P} \Delta P$$

Where $\frac{\partial W}{\partial P}$ is the Jacobian of the wrap:

$$\frac{\partial W}{\partial P} = \begin{pmatrix} \frac{\partial W_x}{\partial p_1} & \dots & \frac{\partial W_x}{\partial p_n} \\ \frac{\partial W_y}{\partial p_1} & \dots & \frac{\partial W_y}{\partial p_n} \end{pmatrix}$$

$$\nabla I_2 = (I_x \quad I_y)$$

$$\nabla I_2 \frac{\partial W}{\partial P} = \begin{pmatrix} I_{xx} & I_{xy} & I_x & I_{yx} & I_{yy} & I_y \end{pmatrix}$$

For a 2D affine transformation:

$$W(X; P) = \begin{pmatrix} p1 & p2 & p3 \\ p4 & p5 & p6 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$$\frac{\partial W}{\partial P} = \begin{pmatrix} x & y & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x & y & 1 \end{pmatrix}$$

For translation:

$$W(X; P) = \begin{pmatrix} x + p_1 \\ y + p_2 \end{pmatrix}$$

$$\frac{\partial W}{\partial P} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Now we will organize all the points in the region of interest and get:

$$B = (I_x * X \quad I_x * Y \quad I_x \quad I_y * X \quad I_y * Y \quad I_y)$$

Where the dimensions are $N \times 6$ and N is the number points in the region of interest. In summary:

$$\Delta P^* = \arg \min_{\Delta P} \sum_X [I_2(W(X; P)) + \nabla I_2 \frac{\partial W}{\partial P} \Delta P - I_1(X)]^2$$

$$\Delta P^* = \arg \min_{\Delta P} (I_t + B \Delta P)^T (I_t + B \Delta P)$$

$$I_t = I_2(W(X; P)) - I_1(X)$$

The only unknowns here are ΔP , and we will solve this with LS to get:

$$\Delta P^* = -(B^T B)^{-1} B^T I_t$$

For 2D translation:

$$\frac{\partial W}{\partial P} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

$$\nabla I_2 \frac{\partial W}{\partial P} = \begin{pmatrix} I_x & I_y \end{pmatrix}$$

$$B = \begin{pmatrix} I_x & I_y \end{pmatrix}_{N \times 2}$$

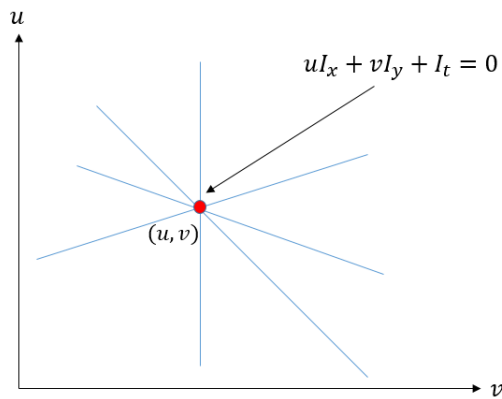
$$\Delta P^* = -(B^T B)^{-1} B^T I_t$$

$$\Delta P^* = - \begin{pmatrix} I_x^T I_x & I_x^T I_y \\ I_y^T I_x & I_y^T I_y \end{pmatrix}^{-1} \begin{pmatrix} I_x^T I_t \\ I_y^T I_t \end{pmatrix}$$

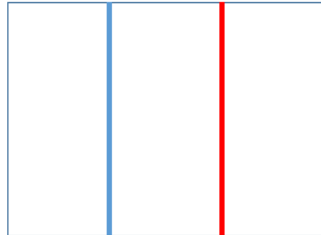
Lastly, our basic equation is of the form:

$$u I_x + v I_y + I_t = 0$$

In the parameter space, every equation is a line and the solution common to all the lines will be the crossing point:



Let's assume we have a static camera. At some time t we will see the blue line and at $t+1$ we will see the red line:

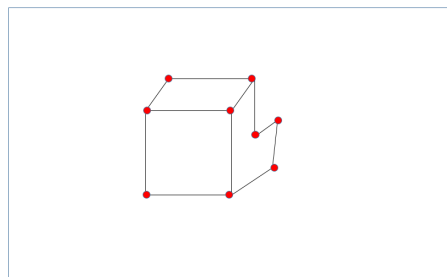


The object is clearly moving to the right, but we cannot determine whether it is also moving diagonally. In other words, we do not know which point along the constraint line corresponds to the true motion. This phenomenon is known as the **aperture problem**.

If the window is small, we can only estimate the motion component perpendicular to the image gradient. The trade-off is that when the window is small, the aperture problem becomes more severe. When the window is larger, however, the simple 2D translation model may no longer be sufficiently expressive, and a more complex model, such as a 2D affine transformation, may be required. The difficulty is that a large window may also include parts of the background, making it harder for the model to handle multiple motions properly.

2.5 Harris Corner Detector

The task back then was finding points of interest and then tracking the object in the next time step.



The question is how to find interest points in an image. Typically, we look for points that are distinct from their neighbors. For example, a white dot on a white background is not a point of interest. Points of interest are usually corners of objects, where there are strong intensity changes in both the x and y directions.

Mathematically, in the case of 2D translation, the matrix $B^T B$ can either be a degree of 0, 1 and 2. If its rank is 0, the point is not distinct from its neighbors. If the rank is 1, the point lies on an edge. If the rank is 2, the point corresponds to a corner.

Harris used this idea in his work. After detecting interest points, he applied the Lucas–Kanade method on a small window around each point to track it in the next frame. We will start by defining an error function:

$$E(u, v) = \sum_{(x,y) \in W} [I(x+u, y+v) - I(x, y)]^2$$

By using Taylor series approximation:

$$I(x+u, y+v) = I(x, y) + uI_x(x, y) + vI_y(x, y)$$

We can insert that to E and get:

$$E(u, v) = \sum_{(x,y) \in W} [(I_x(x, y) \quad I_y(x, y)) \begin{pmatrix} u \\ v \end{pmatrix}]^2$$

$$E(u, v) = \begin{pmatrix} u & v \end{pmatrix} \underbrace{\sum_{(x,y) \in W} \begin{pmatrix} I_x^2 & I_x I_y \\ I_y I_x & I_y^2 \end{pmatrix}}_H \begin{pmatrix} u \\ v \end{pmatrix}$$

What brings this expression to minimum is the eigenvector (u,v) of H corresponding to the smallest eigenvalue.

The entire algorithm can be summarized with the following steps:

1. $(I_x \quad I_y) = \text{grad}(I)$
2. $I_{xx} = I_x * I_x, I_{xy} = I_x * I_y, I_{yy} = I_y * I_y$
3. $S_{xx} = \text{conv}(I_{xx}, g), S_{xy} = \text{conv}(I_{xy}, g), S_{yy} = \text{conv}(I_{yy}, g), g = \text{ones}(5, 5)$
4. We can calculate the image response R in a couple of methods:
 - a. smallest eigenvector: $R = \frac{1}{2}(S_{xx} - S_{yy}) - \sqrt{4S_{xy}S_{xy} + (S_{xx} - S_{yy})^2} = \lambda_{\min}$
 - b. $R = \frac{\lambda_{\min}\lambda_{\max}}{\lambda_{\min} + \lambda_{\max}} = \frac{\text{determinant}(H)}{\text{trace}(H)}$
 - c. $R = S_{xx} * S_{yy} - S_{xy} * S_{xy} - \alpha(S_{xx} + S_{yy})^2 = \lambda_{\min}\lambda_{\max} - \alpha(\lambda_{\min} + \lambda_{\max})^2$
- Where $\alpha = 0.04 \sim 0.06$
5. $R(R < \theta) = 0$
6. Return maximum R in each tile.

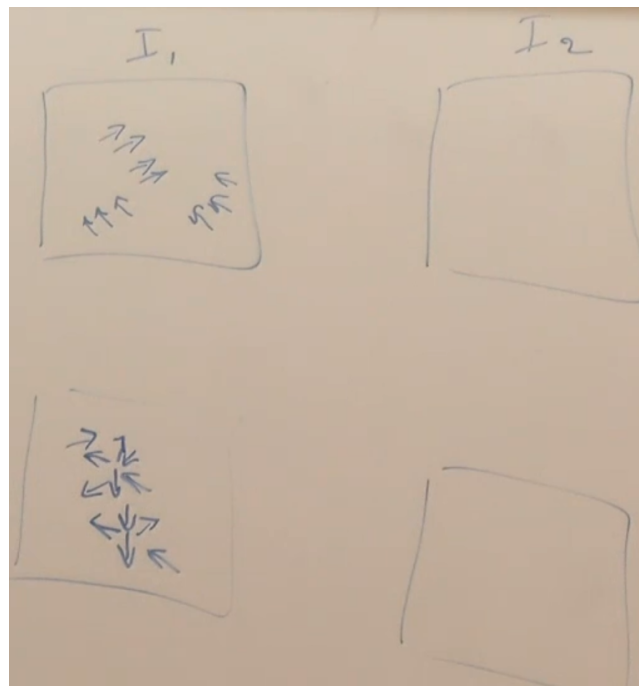
2.6 Applications

The question now is: when should we use Lucas-Kanade (LK) and when should we use Harris Point Detection (HPD)? There are applications where LK alone is sufficient, such as video stabilization or tracking large objects. However, when we need to detect and track points of interest, the process typically begins with HPD in the first frame to identify interest points. A small window is placed around each point, and LK is then used to track them in subsequent frames.

Instead of obtaining a set of images, we obtain a set of points and their positions in each frame. This approach is useful for applications such as motion reconstruction and 3D reconstruction (e.g., SLAM — Simultaneous Localization and Mapping). In such cases, we initialize points with HPD and track them with LK. If a point of interest fails to satisfy the conditions for inverting the relevant matrix, that point is discarded and a new point is selected from its surrounding neighborhood, typically using the Harris detector.

2.7 Horn-Shunck

This algorithm tries to solve the problem of LK but in a more general manner. HS asks where every pixel in the first image was transferred to the 2nd image. The following figure describes (u,v) of every pixel:



The image in the first row is more meaningful. This is because there is an underlying assumption that neighboring points behave similarly. In the Lucas–Kanade (LK) method, this was handled by averaging over a small window around each point. In Horn–Schunck (HS), this idea is taken one step further by incorporating the assumption as a prior in the solution. The HS equation is then given by:

$$(u, v) = \arg \min_{u, v} \int \int [(I_x u + I_y v + I_t)^2 + \alpha(|\nabla u|^2 + |\nabla v|^2)] dx dy$$

$$|\nabla u| = \left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial u}{\partial y}\right)^2 = u_x^2 + u_y^2$$

The first term is the Taylor series term we saw earlier and is called the **data term**. Here, we are solving for u and v , which describe the motion in the image. The innovation in Horn–Schunck lies in the second term, which introduces a smoothness constraint, ensuring that the solution varies as smoothly as possible across the image.

The difference between Lucas–Kanade (LK) and Horn–Schunck (HS) is analogous to the difference between Maximum Likelihood (ML) and Maximum A-Posteriori (MAP) estimation. In LK, the ML approach assumes that the data alone dictates the solution, without any prior knowledge. In HS, on the other hand, prior knowledge is incorporated. The second term in the HS formulation is therefore called the **prior/regularizer** or **smoothness term**. The parameter α controls the relative influence of the data term versus the smoothness term in the solution. Additionally, HS still relies on the same assumptions as LK.

We will use Euler-Lagrange equation:

$$S = \int \int L(x, y, f, f_x, f_y) dx dy$$

Where L is a functional. S gets to minimum if f accomplishes the partial differential equation:

$$\frac{\partial L}{\partial f} = \frac{\partial}{\partial x} \frac{\partial L}{\partial f_x} - \frac{\partial}{\partial y} \frac{\partial L}{\partial f_y} = 0$$

We will now insert the equations to get:

$$L(u, v, u_x, u_y, v_x, v_y) = (I_x u + I_y v + I_t)^2 + \alpha(u_x^2 + u_y^2 + v_x^2 + v_y^2)$$

$$\frac{\partial L}{\partial u} = 2(I_x u + I_y v + I_t)I_x$$

$$\frac{\partial L}{\partial u_x} = 2\alpha u_x$$

$$\frac{\partial}{\partial x} \frac{\partial L}{\partial u} = 2\alpha u_{xx}$$

The same will be with y, and we will do the same for v. The final equations will be:

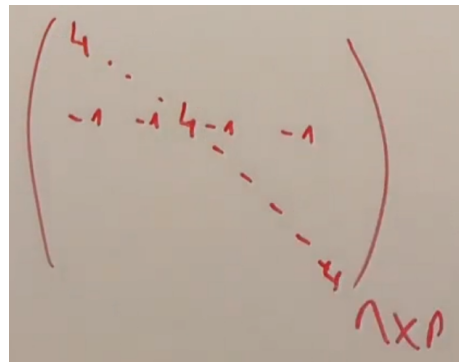
$$\begin{aligned}(I_x u + I_y v + I_t) I_x - \alpha(u_{xx} + u_{yy}) &= 0 \\ (I_x u + I_y v + I_t) I_y - \alpha(v_{xx} + v_{yy}) &= 0\end{aligned}$$

We can write those equation in a vector form. We will use a trick where we can calculate $u_{xx} + u_{yy}$ with the laplacian: $\begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix}$

Now we can insert and get:

$$\begin{pmatrix} I_x^2 + \alpha L & I_x I_y \\ I_x I_y & I_y^2 + \alpha L \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} I_x I_t \\ I_y I_t \end{pmatrix}$$

The dimensions of u and v are $2N \times 1$. The same will be with the right term of the equation. The matrix here is the same as in LK and HPD, but the dimensions here are a bit trickier. The matrices $I_x^2, I_y^2, I_x I_y$ are diagonal matrices of $n \times n$. The whole matrix will have dimensions of $2n \times 2n$ (a very large matrix). The interesting term is the regularizer αL where L is the Laplacian operator in matrix form and in size $n \times n$. It will take the form of:



$$\begin{pmatrix} 4 & & & & & & \\ & -1 & & & & & \\ & & 4 & & & & \\ & & & -1 & & & \\ & & & & 4 & & \\ & & & & & -1 & \\ & & & & & & 4 \end{pmatrix} \quad 7 \times 7$$

At every row we will have -1 in different locations which match to the neighbors up, down, left and right. This is a very wasteful way of writing the filter of 3×3 earlier.

In matrix form:

The image shows a handwritten matrix equation. On the left, a vector U is multiplied by a 3×3 matrix:

$$U * \begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix} =$$
 This is followed by a large $n \times n$ matrix in parentheses, which is a sparse Laplacian matrix. Its diagonal elements are 4, and its off-diagonal elements are -1, forming a banded structure. Below this matrix is the label $n \times n$. To the right of the Laplacian matrix is a vector U in parentheses, with the label $n \times 1$ below it.

We want the Laplacian to be as little as possible so the solution won't be that far of from the solution without the regular term. That way, the solution will overall fulfill both constraints: the data term and the regularization. The set of equations here can be written as $Ax = b$ and is very sparse. We have at every row $1+5+1=7$ elements in a row instead of n and it makes it a very fast solution.

3 Applications of Motion Estimation

We will go over the 3 applications:

1. **Super Resolution** - We are given a low-resolution video and aim to produce a high-resolution version. To begin, we will focus on a simpler problem: generating a single high-resolution image from a set of low-resolution frames. For example, we could take the first five frames, combine them to create one high-resolution frame, and then repeat the process for the next five frames.

2. **Hyperlapse** - This problem addresses video stabilization in a challenging context. When recording a video with a modern smartphone, the footage is often longer than desired, and we may want to produce a shorter version. One approach is to sample frames at regular intervals, but this often results in a jittery video that is unpleasant to watch. Instead, we aim to generate a smooth, condensed version of the video.

3. **Motion Magnification** - The task is to detect very small motions, even those smaller than a single pixel. Lucas-Kanade (LK) and Horn-Schunck (HS) cannot handle such sub-pixel motions, but other approaches have successfully addressed this challenge.

3.1 Super Resolution

We are given a series of low resolution images $(Y_i)_{i=1}^n$ and we search for HR image "x" which will describe $(Y_i)_{i=1}^n$. WE first need to define a model which takes this series of LR images and outputs one HR image.

The model we will take will be:

$$Y_{i[n^2 \times 1]} = D_i H_i F_i X_{[r^2 n^2 \times 1]}$$

Where:

X - HR image which was upscaled by r in both height and width. This image was then converted to a 1-D vector.

F - Motion - We believe the camera moves a bit between frames so F estimates it's motion. Its dimensions are $r^2 n^2 \times r^2 n^2$

H - Blur - The camera may blur the image a bit. In our case, we will use gaussian blur for example. Its dimensions are also $r^2 n^2 \times r^2 n^2$

D - Decimation. The dimentions are $n^2 \times r^2 n^2$

Y_i - LR frame converted to a 1-D vecotr.

We will use the following assumptions:

1. H - the blur kernel is known and fixed for all the frames. We will use gaussian blur here.
2. D - the sampling matrix is known and fixed to all the frames. It will simply take every r element

3. F_i - is calculated on the frames $(Y_i)_{i=1}^n$, where F_1 is the identity.
Using those assumptions we got:

$$Y_i = DHF_i X$$

$$\begin{pmatrix} Y_1 \\ \vdots \\ Y_k \end{pmatrix} = \begin{pmatrix} DHF_1 \\ DHF_2 \\ \vdots \\ DHF_k \end{pmatrix} X$$

This can be written in the form:

$$b = AX$$

From here we can use linear operations to find X.

We can look at this from a statistical approach:

$$Y_i = DHF_i X + v_i$$

$$v_i \sim N(0, \sigma^2)$$

There are two methods to solve this:

1. Maximum Likelihood:

$$x = \arg \min_x \sum_{i=1}^k \|DHF_x X - Y_i\|_2^2$$

$$\epsilon_{ML}^2(x) = \sum_{i=1}^k \|DHF_x X - Y_i\|_2^2$$

$$\frac{\partial \epsilon_{ML}^2(x)}{\partial X} = 2 \sum_{i=1}^k F_i^T H^T D^T (DHF_i X - Y_i) = 0$$

$$\sum_{i=1}^k F_i^T H^T D^T DHF_i X = \sum_{i=1}^k F_i^T H^T D^T Y_i$$

Which can also be written as:

$$AX = B$$

In this case we have a statistical explanation for this. However, those matrices are huge, so we will solve them iteratively:

$$X_{n+1} = X_n - \beta \sum_{i=1}^k F_i^T H^T D^T (DHF_i X_n - Y_i)$$

Each one of those operations don't need to be described with huge matrices but with functions implemented by code.

In addition, we can notice here the error in LR is $DHF_i X_n - Y_i$ and the backproject to HR is $F_i^T H^T D^T$. This problem is linear so it will converge to the solution.

2. Maximum A-Posteriori (MAP) - the most acceptable solution

Instead of calculating $Pr(Y|X)$ we will calculate $Pr(X|Y)$. From Bayes:

$$P(X|Y) = \frac{P(Y|X)P(X)}{P(Y)}$$

Here, $P(Y|X)$ is the likelihood which describes how good is X in describing the data Y. $P(X)$ is the prior and describes how legitimate is X for the solution. Thus, we want to maximize the posterior $P(X|Y)$ and not the likelihood $P(Y|X)$ because the X which describes best the data Y is not legitimate. In other words, we want X to look like a real image and describes best the data Y. This is similar to how we worked with HS.

We got:

$$x = \arg \max_X P(X|Y) = \arg \max_X P(Y|X)P(X)$$

Assumption: The distribution function of the images is assumed to be Gibbs distribution:

$$P(x) = Ce^{-A(x)}$$

$$X_{map} = \arg \min_X ||DHF X - Y||_2^2 + \lambda A(x)$$

We would also define what is the functional $A(x)$. An example for $A(x)$ would be the Laplacian $\begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix}$ and we want to find X for which $||Lx||$ is small. In other words, the image is smooth to fragments. The final solution will be:

$$X_{n+1} = X_n - \beta \sum_{i=1}^k F_i^T H^T D^T (DHF_i X_n - Y_i) + \gamma L(X_n)$$

Where $\gamma L(X_n)$ is a penalty for not getting a legitimate image.

3.2 Hyperlapse

Let's consider an example: a person running while recording a video. We want to take this long video and sample frames at regular time intervals. Simply doing this would produce a jittery video, which we want to smooth. Suppose this represents the running route, and we aim to sample frames at each time interval:



The problem is that the motion vector can vary significantly from frame to frame, making video stabilization challenging. Initially, the solution was to use the available frames and stabilize the video based on the best possible reference. Later, it was discovered that it is not necessary to sample frames at constant time intervals; instead, selecting frames where the camera is oriented in approximately the same direction produces better results.

We will do the following steps:

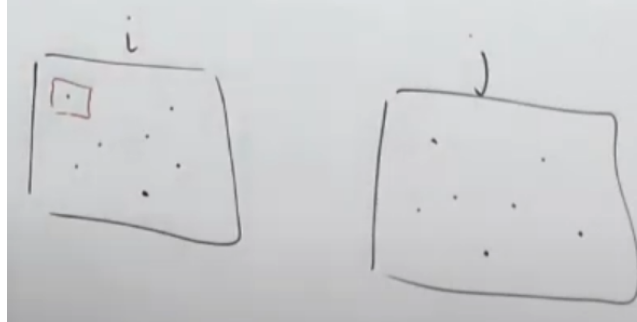
1. We will define cost function C which will measure the matching between a pair of images.
 2. We will build matrix "Dynamic Programming" and find the optimal path.
- The cost function between two images will be:

$$C_r(i, j) = \frac{1}{n} \sum_{p=1}^n ||(x_p, y_p)_j^T - T(i, j)(x_p, y_p)_i^T||_2$$

$C_r(i, j)$ - the cost of geometrical match between image i and image j

$T(i, j)$ - the transformation matrix which is calculated

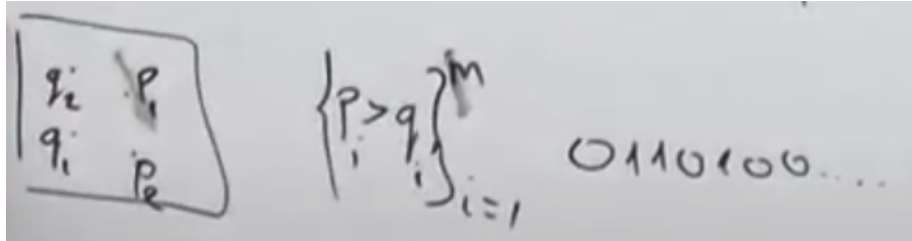
The way they do that is the following:



1. Find points of interest - use Harris in each image separately.
2. Describe points of interest. The most naive approach is open a window of 5×5 and do cross correlation of the patch with every points of interest and find the one which minimize the error. Instead, we used BRIEF algorithm.
3. Match points of interest - calculate Hamming distance between BRIEF descriptors.
4. Calculate $T(i, j)$. Specifically, they used 2D Affine.

BRIEF algorithm

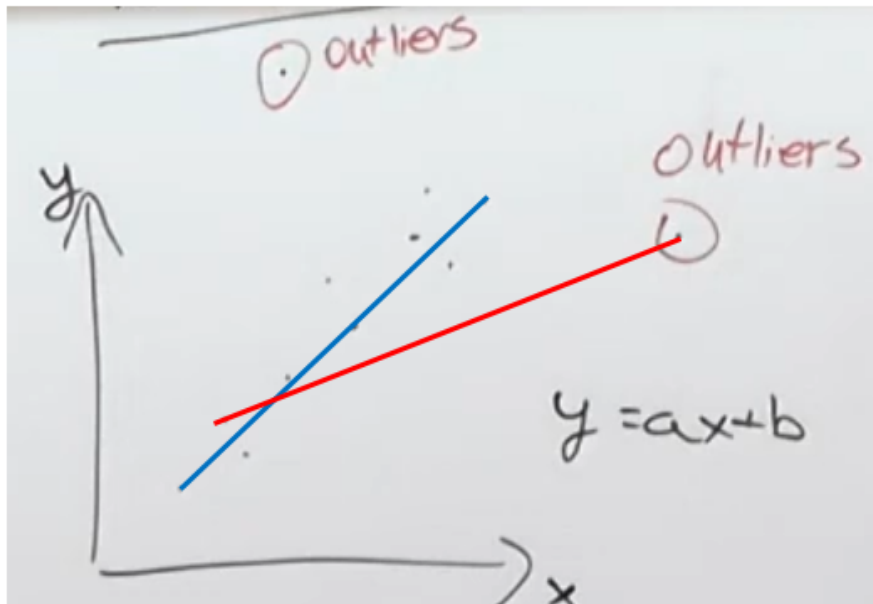
We build a series of pixels and ask:



We define a window with a fixed size and in a fixed positions if $p_i > q_i$ and create a binary string which describes the patch. This description is resilient to small changes in the brightness and positions. This also gets the results very fast.

RANSAC

When calculating transformations, there is a risk of matching false pairs of points. The RANSAC algorithm is designed to address this problem. A common way to explain RANSAC is through line fitting. Suppose we have a set of points and want to estimate the parameters of a line:



The main problem here is the presence of outliers—false points that are incorrectly identified by the algorithm and negatively affect the solution. RANSAC is designed to eliminate these outliers.

First, we must determine how many points are needed to solve the model. In the case of line fitting, only two points are required. We start by selecting two points and solving for the model parameters. The problem arises if one of these points is an outlier: in that case, most of the other points will not satisfy the solution. RANSAC addresses this by randomly selecting points many times and identifying the model that is consistent with the largest number of points.

In our case, for a 2D affine transformation, we need only three pairs of matching points. Using RANSAC, we randomly select three pairs of matching points and, for each set, determine how many points are consistent with the resulting transformation. A point is considered consistent if, after applying the transformation to a point that was not used in RANSAC from image i , the distance between the transformed point and its corresponding point in image j is below a given threshold.

Now the question is How many times to we need to randomly choose 3 pairs.

Assume:

W - the fraction of inliers in the data

P - the probability to sample inliers

n - the minimal number of points to solve the model

t - the threshold for a point to be considered an inlier

d - the minimal number of points to support the model

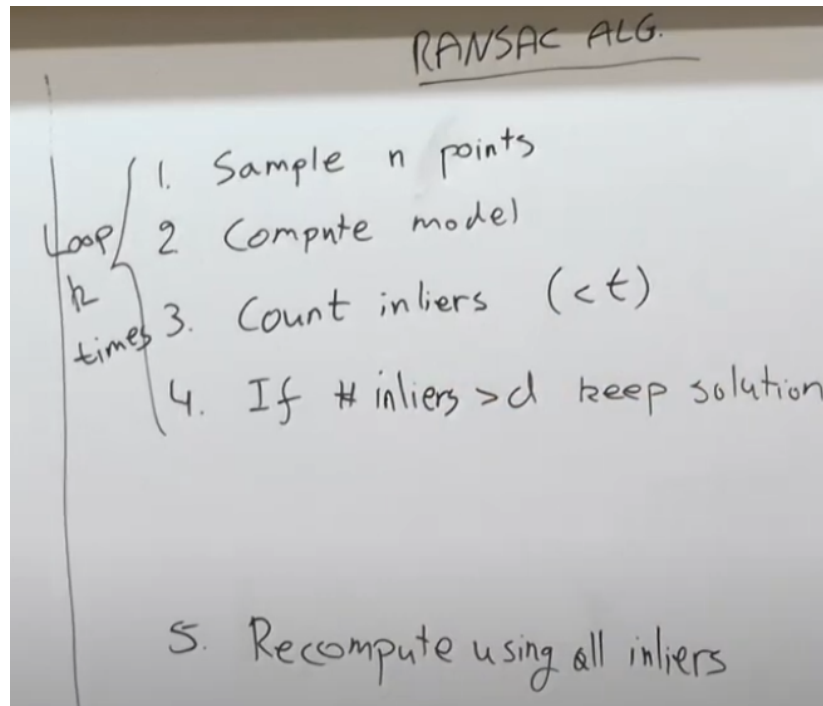
We need to find K - the number of draws required

The probability all the points are inliers is W^n . Thus, the probability at least one is outlier is $1 - W^n$. The probability of K failures will then be $(1 - W^n)^K$. Thus, we want:

$$1 - P = (1 - W^n)^K$$

$$K = \frac{\log(1 - P)}{\log(1 - W^n)}$$

In summery, RANSAC algorithm would be:



Adjustments to Hyperlapse solution

1. Geometric index

We will define:

$$C_0(i, j) = \|(x_0, y_0)_j^T - T(i, j)(x_0, y_0)_i^T\|_2$$

$$C_m(i, j) = \begin{cases} C_0(i, j) & \text{if } C_r(i, j) < \theta \\ \gamma & \text{O.W.} \end{cases}$$

$C_0(i, j)$ is distance between the center of image i after transformation to the center of image j .

2. Speedup Error

$$C_s(i, j, \mu) = \min(\|(j - i) - \mu\|_2^2, T_s)$$

This measures the speedup. μ is the desired speedup, and larger the difference between i and j is from the desired sampling rate μ we get larger penalty upto some threshold T_s .

3. Constant speed

$$C_a(h, i, j) = \min(\|(j - i) - (i - h)\|_2^2, T_a)$$

This cost forces a triplet of frames taken to be about constant intervals.

4. Total cost function

$$C(h, i, j, \mu) = C_m(i, j) + \lambda_s C_s(i, j, \mu) + \lambda_a C_a(h, i, j)$$

Hyperlapse Algorithm

HyperLapse Alg.
Input: D - speed up
Output: P - path
Initialization
{ For $i=1$ to g
 For $j=i+1$ to $i+w$
 $D_r(i,j) = C_m(i,j) + \lambda_s C_s(i,j,D)$
 } End
} End

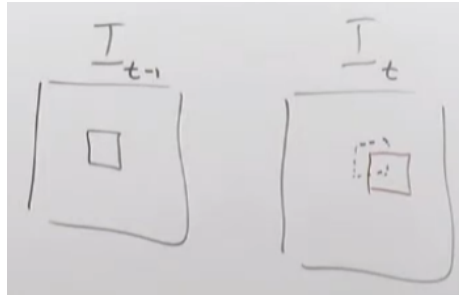
Pass I
For $i=g$ to N
 For $j=i+1$ to $i+w$
 $c = C_m(i,j) + \lambda_s C_s(i,j,P)$
 $D(i,j) = c + \min_k \left[D_r(i-k,i) + \lambda_a C_a(i-k,i,j) \right]$
 $T(i,j) = \arg \min_{k=1}^w \left[D_r(i-k,i) + \lambda_a C_a(i-k,i,j) \right]$
Pass II
 $P \leftarrow \text{Back-Trace}(T)$

4 Kalman and Particle Filters

Typically, we do not have access to the entire video at once, so we need to process it online. In such situations, filtering techniques are the main tools used. We will first motivate their use through the example of a tracking problem.

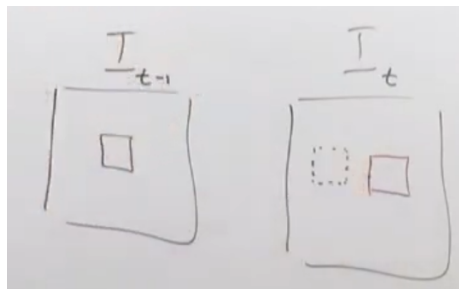
4.1 Tracking

Let us assume we have a collection of frames and a motion estimation algorithm. Suppose an external algorithm has detected an object for tracking. The Lucas–Kanade method requires a template to work with — in our case, the content inside the tracking window from the previous frame — as well as a good initial estimate of the object’s position in the current frame. In the figure below, the ground truth is marked in red, and the initial guess is marked in blue:



One possible initial guess is the position of the object in the previous frame. When using Lucas–Kanade, we assume that the video has a high enough frame rate so that a few iterations starting from this initial guess will allow the algorithm to converge.

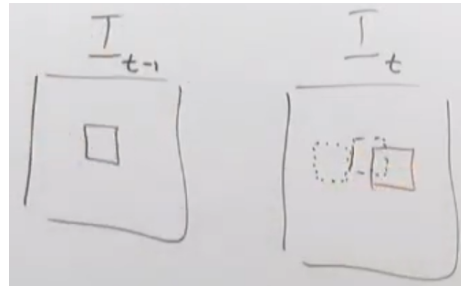
In practice, we do not always have a high frame rate — for example, due to limited CPU performance — and the object may move significantly between frames. In such cases, the initial guess can be far from the ground truth:



In this situation, the Lucas–Kanade method may fail to converge to the correct solution. To address this, we can make assumptions about the motion of the object — assumptions that will later be used by Kalman and Particle filters.

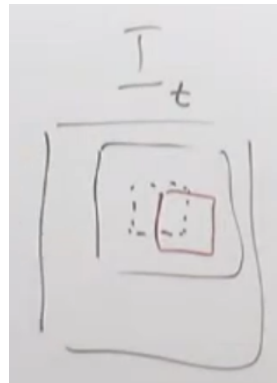
For example, we might assume that the object moves along a straight line, with constant velocity, or with constant acceleration.

Using one of these assumptions, we can predict the object's motion according to the chosen model and refine our initial guess:



In fact, what Lucas–Kanade (LK) does is essentially gradient descent. Suppose this is not what we want. Instead, we may prefer a template matching approach: we take the tracking window from the previous frame, slide it over the current frame, and assign a score at each position based on how well the window matches. In this scenario, we need a Kalman filter for several reasons:

1. **Computation efficiency:** By predicting the object's position in the current frame, we do not need to perform template matching over the entire frame; instead, we can search within a smaller, localized area:



2. **Object ambiguity:** There may be multiple objects in the frame that look similar — for example, several white cars. In such cases, template matching alone can easily fail to distinguish between them. However, a Kalman filter prediction provides an estimate of the desired object's position, helping to eliminate other candidates.

3. **Appearance space:** There are cases where an object's appearance may change drastically between frames — for example, due to changes in orientation, lighting, or brightness. In such situations, template matching should not rely solely on the tracking window from the previous frame. Instead, we can create a template that better represents the object's appearance over time. One approach is to average N tracking windows (i.e., compute the average of several images), which is more robust to drastic changes and provides a better representation of the object accumulated over time. Since this involves integrating information over time, using a Kalman filter is particularly effective in this context.

4.2 Kalman Filter

The following are the main equations of Kalman Filter:

The image shows handwritten notes on a piece of paper, divided into two columns by a vertical line. The left column is titled 'Project' and the right column is titled 'Update'. Above the 'Update' column, there is a red bracket labeled 'Kalman Gain' and the handwritten text 'inf pon'. The equations are as follows:

Project stage:

$$\hat{x}_{k+1} = \Phi \hat{x}_k$$
$$P_{k+1} = \Phi P_k \Phi^T + Q$$

Update stage:

$$K_k = P_k' H^T (H P_k' H^T + R)^{-1}$$
$$\hat{x}_k = \hat{x}_k' + K_k (z_k - H \hat{x}_k')$$
$$P_k = (I - K_k H) P_k'$$

Kalman Filter is divided by two stages: project and update.

Project stage

In the project stage, we used all the information of past frames we could accumulate and now we project this information to the new frame. The vector $\hat{x}_k$ has all the information we need (for example the position, velocity, etc). The matrix Φ predicts the state at time $k + 1$ based on the time k .

P_{K+1} is the covariance matrix of x at time $k+1$.

Update stage

In the update stage, we go over the information received from the current frame and check whether it agrees with the information of past frames. Mathematically, we update the vector $\hat{x}_k$ and its covariance based on the information of the current frame. This is done by what is called "**Kalman Gain**". It is the result of the optimal solution we can get based on gaussian assumptions and independence of random variables which will be shown later.

In the tracking example, the initial guess is computed based on past frames. We then apply Lucas-Kanade (LK) or template matching on the current frame to estimate the object's current position. The final result is an integration of the initial guess and the measurement from LK or template matching. If we place more trust in the motion model, the result will be closer to the initial guess; if the current frame provides more reliable information, the result will be closer to the LK or template matching estimate.

Before we start with developing the equations, we will use the following notations:
 $\hat{x}_k$ - **Posterior state vector**. This is the final result of Kalman Filter after the processing of frame k.

$\hat{x}'_k$ - **Prior state vector**. This is the prediction, or the initial guess, based on the past frames.

$\hat{z}_k$ - **Measurement vector**. This is the output of LK based on the current frame.

Equation Development

Let's assume:

$$x_{k+1} = \Phi x_k + w_k$$

Where: $x \in R^n$, $\Phi \in R^{n \times n}$, $w \sim N(0, \sigma^2)$.

x_{k+1} is a vector with all the information we want about the world - position, velocity, acceleration, etc. We have a known matrix Φ which transforms the state x_k to state x_{k+1} . w_k is the noise of the model itself. For example, suppose the object we are tracking becomes occluded for 10 frames. We can continue to track it using the motion model's prediction, but without measurements from the current frames, the uncertainty grows over time, and the tracking error accumulates.

Now, let's define the measurement vector:

$$z_k = Hx_k + v_k$$

Where: $z \in R^m$, $H \in R^{m \times n}$, $v \sim N(0, \tilde{\sigma}^2)$.

x_k has all the information of the object (for example, if we have position and velocity then we have 4 parameters). The measurement itself doesn't necessarily need all the information. For example, the measurement can be only the position and thus it will have only 2 parameters. H therefore is linear relation

between x_k and z_k . v_k is the noise of the measurements (we assume it has normal distribution).

We assume the noise of the process and the noise of the measurements are independent:

$$E[w_k v_j^T] = 0$$

We will define the **System noise** as:

$$Q = E[w_k w_k^T]$$

We will define the **Measurement noise** as:

$$R = E[v_k v_k^T]$$

We will define the error to be:

$$e_k = x_k - \hat{x}_k$$

Where x_k is the ground truth and $\hat{x}_k$ the approximation. Usually, the GT is not given to us, so in Kalman filter we assume the position given in the first frame is the GT (someone gave it to us), and we assume an invariant phenomenon where we try to preserve the optimal position given until now we were at the optimal position.

The covariance matrix would be:

$$P_k = E[e_k e_k^T] = E[(x_k - \hat{x}_k)(x_k - \hat{x}_k)^T]$$

Our task is to minimize P (or more accurately, it's eigen values). We emphasize that we want the mean error on the entire frames is minimized, and not just in a single frame. We can visualize this as an ellipse of uncertainty, and we want it to be small as possible,

We want:

$$\hat{x}_k = \hat{x}'_k + K_k(z_k - H\hat{x}'_k)$$

We insert $z_k = Hx_k + v_k$ and get:

$$\hat{x}_k = \hat{x}'_k + K_k(Hx_k + v_k - H\hat{x}'_k)$$

We will insert to the definition of P and get:

$$P_k = E[[(I - K_k H)(x_k - \hat{x}'_k) - K_k v_k][(I - K_k H)(x_k - \hat{x}'_k) - K_k v_k]^T]$$

We simplify the expression and get:

$$P_k = (I - K_k H)P'_k(I - K_k H)^T + K_k R K_k^T$$

Where P'_k is the covariance before the update, R is the measurement error and P_k is the weighted average of both of them. If K is 0, we don't have measurement error (we don't use the measurement of the current frame). If K is equal to 1, then the first element will be 0 and we will only have the measurement error.

Kalman suggested finding K which minimize $Trace[P_k]$. This is because trace is the sum of all the elements in the main diagonal, and in $E[e_k e_k^T]$ those are exactly the variances of each variable in our vector state x . Everything is known to us except K , and this is what we want to find. This is done by differentiating and finding K which makes the derivative 0.

We will get:

$$T[P_k] = T[P'_k] - 2T[K_k H P'_k] + T[K_k (H P'_k H^T + R) K_k^T]$$

$$\frac{\partial T[P_k]}{\partial K_k} = -2(H P'_k)^T + 2K_k (H P'_k H^T + R) = 0$$

And we got the **Kalman Gain**:

$$K_k = P'_k H^T (H P'_k H^T + R)^{-1}$$

Now we will calculate P_k :

$$P_k = P'_k - P'_k H^T (H P'_k H^T + R)^{-1} H P'_k = P'_k - K_k H P'_k = (I - K_k H) P'_k$$

As shown before. In a private case, let's assume for now that everything is 1-D. We will get in this case:

$$K = \frac{P'}{P' + R}$$

We can see that when $R \rightarrow 0$ (the measurement error goes to 0), then $K \rightarrow 1$. In that case, we believe in our measurement and the new state will be solely influenced by the measurement. In other case, if $R \rightarrow \infty$, then $K \rightarrow 0$ and the new state is only calculated based on the prediction of the model.

By definition:

$$\hat{x}'_{k+1} = \Phi \hat{x}_k$$

$$e'_{k+1} = x_{k+1} - \hat{x}'_{k+1} = (\Phi x_k + w_k) - \Phi \hat{x}_k = \Phi e_k + w_k$$

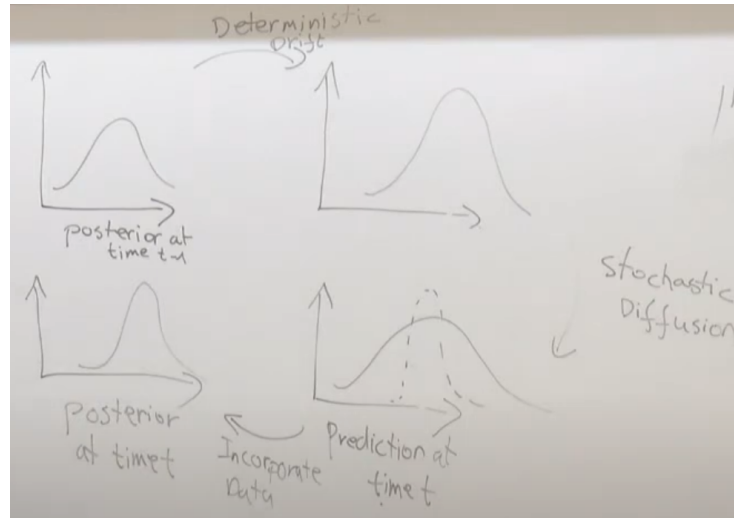
We will get:

$$P'_{k+1} = E[e'_{k+1} (e'_{k+1})^T] = E[(\Phi e_k + w_k)(\Phi e_k + w_k)^T]$$

Let's assume w_k, e_k are independant and we will get:

$$P'_{k+1} = E[e'_{k+1} (e'_{k+1})^T] = E[(\Phi e_k)(\Phi e_k)^T] + E[w_k w_k^T] = \Phi P_k \Phi^T + Q$$

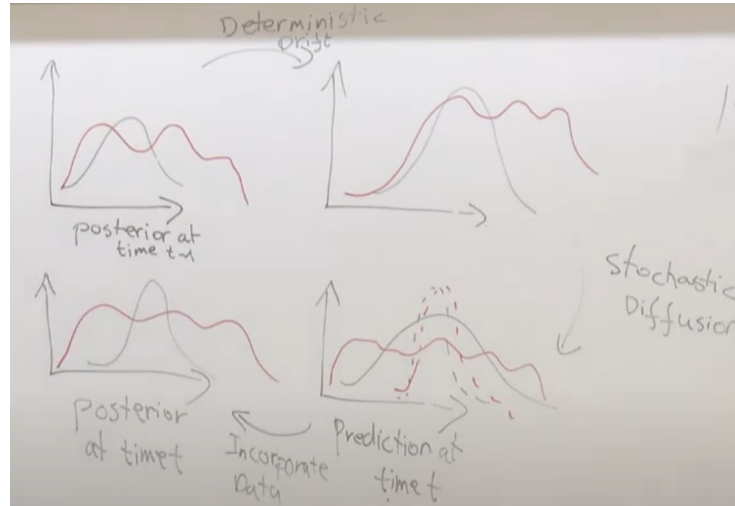
The following diagram explains visually how Kalman filter works:



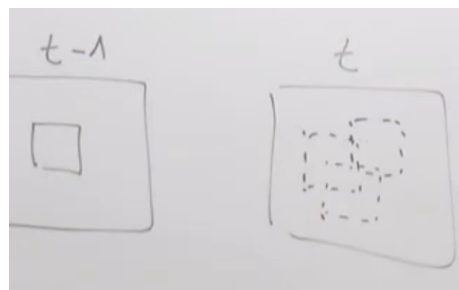
We start with a posterior at time $t-1$. The x-axis is the state vector x (in this case it's scalar), and the y-axis is the probability of each state vector. We have the deterministic drift which is our Φ . It is the prediction of the current state based on past states. The stochastic diffusion adds error, flattening the gaussian. In addition, we get the measurement of the current time t (the dashed gaussian). Then we incorporate the data of both gaussians to get posterior at time t .

4.3 Particle Filter

Particle filters want to do the same as Kalman filter, but without gaussian assumptions. The diagram would be similar:



The distribution on Particle Filter (the red one in the diagram) would no longer be a gaussian. Solving tracking problem with a Particle Filter will be in the following way:



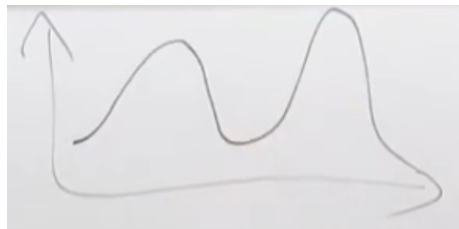
We have a probability distribution over the state space. To represent this distribution, we randomly generate particles — discrete states where the object could be located (typically 100–200 particles). The particles are sampled according to the distribution function, and we need enough particles so that their distribution closely approximates the underlying probability distribution.

During stochastic diffusion, we reduce the probability of each particle to account for uncertainty in the model. Next, we compute the posterior by evaluating how well each particle (tracking window) matches the template. This is done by calculating a matching score for each particle, which measures its similarity to the template. These scores represent the likelihood of each particle.

Then, we compute the posterior by combining the prior and the likelihood. For example, if the model predicts that the object is at a certain tracking window (the prior), but the image in that window does not match the template (the likelihood), then the posterior probability for that particle will be low.

Finally, we select the particle with the highest posterior probability. In the Kalman filter, the maximum of the Gaussian is straightforward to compute—it is simply the mean. In a particle filter, however, determining the most likely state is less direct, and there are several approaches, such as using a weighted average of the particles or selecting the particle with the maximum posterior.

To explain the tradeoff between the maximum and the average, we will use the following figure:



When selecting the particle with the maximum posterior, if two objects have similarly high probabilities, the algorithm may jitter between them, resulting in an unstable estimate. On the other hand, using a weighted average produces a more stable result, but it may sometimes yield an incorrect estimate if the average does not correspond to an actual object location.

Equation development

We will define:

x_t - state vector at time t .

$x_{1:t}$ - $(x_1, \dots, x_t)$ the history.

z_t - the measurement at time t .

$z_{1:t}$ - $(z_1, \dots, z_t)$ the history of measurements.

One thing we can be interested is the posterior distribution $P(x_{1:t}|z_{1:t})$. However, we will work with **filter distribution** $P(x_t|z_{1:t}) = \int_{x_1} \dots \int_{x_{t-1}} P(x_{1:t}|z_{1:t})$.

Lately, we will work with the likelihood:

$$P(x_{1:t}|z_{1:t}) = \frac{P(z_{1:t}|x_{1:t})P(x_{1:t})}{P(z_{1:t})}$$

We will use the following assumptions for simplicity:

1. Markov assumption:

$$P(x_t|x_{1:t}) = P(x_t|x_{t-1})$$

This assumption was also used in Kalman filter. Therefore:

$$P(x_{1:t}) = P(x_t|x_{t-1})P(x_{1:t-1}) = p(x_1) \prod_{j=2}^t P(x_j|x_{j-1})$$

In the case of Kalman, the matrix Φ would be $P(x_t|x_{t-1})$.

2. Conditional Independance:

$$P(z_{1:t}|x_{1:t}) = P(z_t|x_t)P(z_{1:t-1}|x_{1:t-1}) = \prod_{\tau=1}^t P(z_\tau|x_\tau)$$

Now we will develop the equations:

$$P(x_{1:t}|z_{1:t}) \propto P(z_{1:t}|x_{1:t})P(x_{1:t}) = \prod_{\tau=1}^t P(z_\tau|x_\tau) \cdot p(x_1) \prod_{j=2}^t P(x_j|x_{j-1}) \propto P(z_t|x_t)P(x_t|x_{t-1})P(x_{1:t-1}|z_{1:t-1})$$

This expression allows us to describe $P(x_{1:t}|z_{1:t})$ recursively. We can see from here that $P(x_{1:t-1}|z_{1:t-1})$ is the recursion, $P(x_t|x_{t-1})$ is the motion model and $P(z_t|x_t)$ is the likelihood. It can be shown:

$$P(x_t|z_{1:t}) = \int_{x_1} \dots \int_{x_{t-1}} P(x_{1:t}|z_{1:t}) = C \cdot P(z_t|x_t)P(x_t|z_{1:t-1})$$
$$P(x_t|z_{1:t-1}) = \int_{x_{t-1}} P(x_t|x_{t-1})P(x_{t-1}|z_{1:t-1})$$

The final equations are therefore:

Posterior:

$$P(x_{1:t}|z_{1:t}) = B \cdot P(z_t|x_t)P(x_t|x_{t-1})P(x_{1:t-1}|z_{1:t-1})$$

Filtering:

$$P(x_t|z_{1:t}) = \int_{x_1} \dots \int_{x_{t-1}} P(x_{1:t}|z_{1:t}) = C \cdot P(z_t|x_t)P(x_t|z_{1:t-1})$$
$$P(x_t|z_{1:t-1}) = \int_{x_{t-1}} P(x_t|x_{t-1})P(x_{t-1}|z_{1:t-1})$$

Now, we will go over the algorithm of Particle filter.

Condensation

CONDENSATION

Input: $\{S_{t-1}^{(n)}, \pi_{t-1}^{(n)}, C_{t-1}^{(n)}\}_{n=1}^N$

Output: $\{S_t^{(n)}, \pi_t^{(n)}, C_t^{(n)}\}_{n=1}^N$

- 1) Select $S_t^{(n)}$ as follows
 - a) generate r at random $r \in [0, 1]$
 - b) find smallest j $C_{t-1}^{(j)} \geq r$
 - c) Set $S_t^{(n)} = S_{t-1}^{(j)}$
- 2) Predict

$$P(X_t | X_{t-1} = S_t^{(n)})$$

i.e. $S_t^{(n)} = A S_{t-1}^{(n)} + B W_t^{(n)}$
- 3) Measure

$$\pi_t^{(n)} = P(Z_t | X_t = S_t^{(n)})$$

normalize $\sum_n \pi_t^{(n)} = 1$

and let: $C_t^{(0)} = 0$

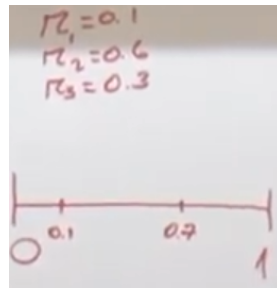
$$C_t^{(n)} = C_t^{(n-1)} + \pi_t^{(n)}$$

Return $\sum \pi_t^{(n)} f(S_t^{(n)})$

S - State vector (which was x)
 C - slack variable (will be later explained).
 π - weight of a particle
 n - the index of the particle

t - timestep index

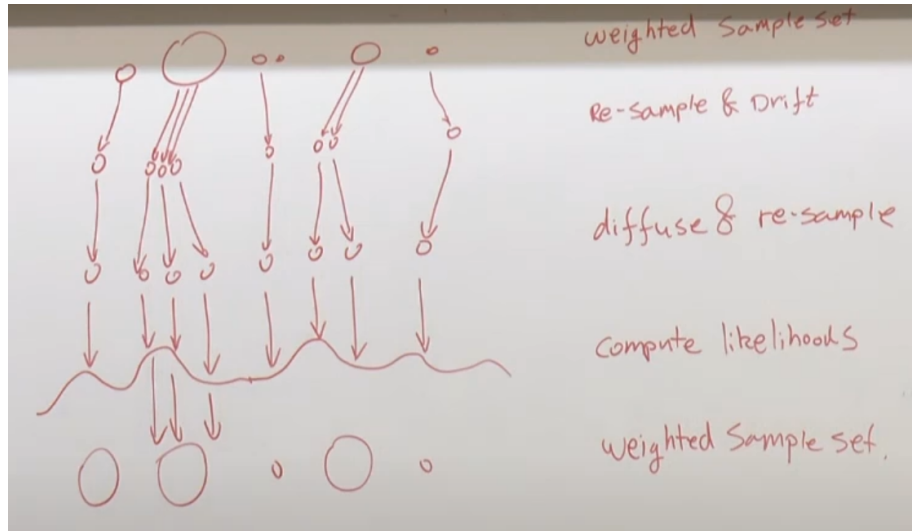
At the select stage, we randomly pick particles. $S_t^{(n)}$ are in fact priors. We will use an example to explain how we randomly sample. Suppose we have 3 particles and we want them to have the following probabilities:



In this case, we desire that we pick the 1st particle 10% of the time, the 2nd particle 60% and the 3rd - 30% of the time. We will do this sampling a random variable r which has a uniform distribution between 0 and 1. If the $0 < r < 0.1$, then we pick particle 1. If $0.1 < r < 0.1 + 0.6$, then we choose the 2nd particle, otherwise the 3rd particle.

The predict stage is in fact the same as the one in Kalman filter but is more general and we do this for every particle n . In the measure stage, we look at the probability which tells that if the state x_t was $S_t^{(n)}$, then how good was this state for describing the data. This is where the similarity function between templates comes in and π will indicate the score. An example of could be calculating the Sum of Square Difference (SSD), and then calculate e^{-SSD} to get a value between 0 and 1. The final returned result is a weighted sum.

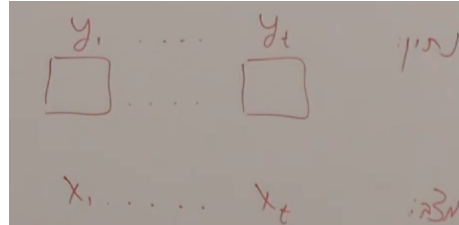
The following diagram describes the algorithm:



We begin with a weighted set of particles at time t_1 and aim to produce a weighted set at time t . The weight of each particle represents its importance: the higher the weight, the higher the probability that the state corresponds to that particle. Next is the resampling stage, where we draw particles deterministically from the original distribution, which is approximated by the current particle set. Particles with large weights may be sampled multiple times, medium-weight particles are sampled once, and very small-weight particles may not be sampled at all. This process is guided by the random variable r described earlier. We then apply drift and diffusion to predict the next state, and finally, we multiply by the likelihood to compute the posterior. The output is the updated weighted particle set, which can be interpreted as a weighted sum representing the state distribution.

5 Texture synthesis

We will start by giving the bottom line of the algorithm and then develop the equations. In order to process a signal in time, we have learned we can use Kalman filter for that. However, Kalman won't be given to us and we will instead try to estimate it's parameters. We are given a series of frames:

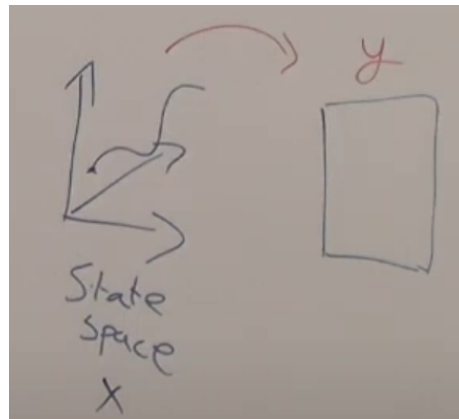


Kalman assumes we have the measurements and the states. They assumed:

$$x_{t+1} = Ax_t + Bv$$

$$y_t = Cx_t + w$$

Where v is a random noise vector. A and x_t are not given, and we aim to estimate them from the observations y_t . In some sense, texture can be viewed as a process in which we use past states to estimate the current state. To prevent the process from being purely deterministic, we introduce noise. This can be visualized as follows:



We define a state space in which the texture evolves over time along a trajectory. Each state along this trajectory is mapped to an image y . The mapping is defined by the second equation introduced earlier: we apply the matrix C and add noise, which makes the generated image y appear more natural. Thus, the first equation represents the motion model in the state space, while the second equation describes the transformation from the state to the observed image, which is the final output we observe.

In order to solve the problem, we represent each frame y as a long vector. If there is a periodic phenomenon between the frames, then constructing a matrix from these frame vectors will result in a matrix whose rank is much smaller than the number of pixels. There are several solutions to this problem. For example, we could apply wavelet decomposition or DCT decomposition, but it has been found that SVD decomposition works best in practice. In other words, the solutions for x_t correspond to the eigenvectors of the matrix described earlier.

We can estimate the process of the texture in time as an ARMA model:

Auto Regressive: $x_t = C + \sum_{i=1}^P p_i x_{t-1} + \epsilon_t$

Where C is constant, p_i are the parameters and ϵ_t is the noise. This can also be written with vectors and a matrix:

$$\begin{pmatrix} x_1 & \dots & x_5 \\ x_2 & \dots & x_6 \\ \vdots & \dots & \vdots \end{pmatrix} \begin{pmatrix} p_1 \\ p_2 \\ \vdots \end{pmatrix} = \begin{pmatrix} x_6 \\ x_7 \\ \vdots \end{pmatrix}$$

Moving Average: $x_t = \mu + \sum_{i=1}^P \theta_i \epsilon_{t-1} + \epsilon_t$

Where μ is the mean value, θ_i are the parameters and ϵ_t are the noise.

Let's go back to our problem: we are given a series of images $(I(t))_{i=1}^T$ and we observe a series of images:

$$y(t) = I(t) + w(t)$$

Let's assume we can model the texture in time with:

$$x(t+1) = Ax(t) + Bv(t)$$

$$y(t) = Cx(t) + w(t)$$

Where $x \in R^n (n \sim 50)$, $y \in R^m (m \sim 200 \times 300)$. There are two noise sources: $v(t)$ is for creating the texture (otherwise we will get a deterministic model which gives a constant behavior), and the second is the noise in the image itself to get a natural look.

To solve the problem, we need to define the error function we need to minimize. One way would be ML estimation: given $y(1) \dots y(T)$, we would like to find:

$$\hat{A}, \hat{B}, \hat{C}, \hat{q} = \arg \max_{A, B, C, q} \log P(y(1) \dots y(T))$$

s.t. the equations and $v(t) \sim q$ (q is a distribution). However, we will use Minimum Mean Error:

$$\hat{A}, \hat{B}, \hat{C}, \hat{q} = \lim_{t \rightarrow \infty} E(\|y(t+1) - \hat{C}\hat{x}(t+1)\|)$$

However, this is also considered a hard problem to solve, so they came up with another approach (under some assumptions) to the problem:

$$x(t+1) = Ax(t) + Bv(t)$$

$$y(t) = Cx(t) + w(t)$$

However, $v(t) \sim N(0, Q)$ and $w(t) \sim N(0, R)$ and the initial condition of x is assumed to be x_0 . **There is not a single solution to this** because if A , B and C are the solution, then given T an invertible matrix, we could replace $A = TAT^{-1}$, $Q = TQT^{-1}$, $C = CT^{-1}$ and $x_0 = Tx_0$ and get an equivalent solution. In our solution we will choose one candidate from the set of solutions.

WE will use SVD decomposition to solve the problem. WE define:

$$Y_1^\tau = [y(1) \dots y(T)] \in R^{m \times \tau}, \tau > n$$

$$X_1^\tau = [x(1) \dots x(T)] \in R^{m \times \tau}, \tau > n$$

$$W_1^\tau = [w(1) \dots w(T)] \in R^{m \times \tau}, \tau > n$$

In matrix form:

$$Y_1^\tau = CX_1^\tau + W_1^\tau$$

$$C \in R^{m \times n}$$

We will demand $C^T C = I$, where I would be called the Identity Matrix. This will restrict the columns will be orthonormal and we chose this constraint to get a single solution from the set of all solutions. From SVD decomposition we will get:

$$Y_1^\tau = U \Sigma V^T$$

$$U \in R^{m \times n}, \Sigma \in R^{n \times n}, V \in R^{n \times \tau}$$

We assumed $w \sim N(0, R)$ so we search $\hat{C}, \hat{x}$ that minimize $\|w_1^\tau\|_F$. We will get:

$$\hat{C}(\tau) = U$$

$$\hat{x}(\tau) = \Sigma V^T$$

To calculate $\hat{A}$ we need to solve:

$$\hat{A}(\tau) = \arg \min_A \|x_1^\tau - Ax_0^{\tau-1}\|$$

Where:

$$X_0^{\tau-1} = [x(0) \dots x(\tau-1)]$$

$$X_1^\tau = [x(1) \dots x(\tau)]$$

The solution will be:

$$\hat{A}(\tau) = \underbrace{\Sigma V^T D_1}_{X_1^\tau} V (V^T \underbrace{D_2 V}_{(X_0^{\tau-1})^{-1}})^{-1} \Sigma^{-1}$$

Where:

$$D_1 = \begin{pmatrix} 0 & 0 \\ I_{\tau-1} & 0 \end{pmatrix}$$

$$D_2 = \begin{pmatrix} I_{\tau-1} & 0 \\ 0 & 0 \end{pmatrix}$$

$$I_{\tau-1} \in (\tau - 1) \times (\tau - 1)$$

6 Layer image representation

In this lecture we will go over two topics: **Layered image representation** and **background subtraction**. What is common to both of them is that we are given a signal (a video or a signal as a function of time), and we want to divide it into basic components.

Layer image representation - introduction

Usually in animated videos, each frame is divided into layers: the first layer is the background, the second contains the characters, etc. Given these layers, we can generate the final video. However, we are interested in the reverse process—given the video, we want to decompose it into different layers.

By using a very basic transformation (for example, Affine), we will see that it is quite easy to model each layer in the video. Most of the time, we can estimate the behavior of very complex scenes using a small number of coherent motions. Usually, a small neighborhood of pixels moves in the same direction rather than each pixel moving independently.

background subtraction - introduction

Given a video, we want to determine for every pixel whether it belongs to a static object or a dynamic object. One simple approach is to observe the intensity of the pixel as a function of time. If it does not change much, we can assume it belongs to a static object.

6.1 Layer image representation

We assume:

$$I_0(x, y) = E_0(x, y)$$

$$I_1(x, y) = I_0(x, y)(1 - \alpha_1(x, y)) + E_1(x, y)\alpha_1(x, y)$$

$$I_k = I_{k-1}(1 - \alpha_k) + E_k\alpha_k$$

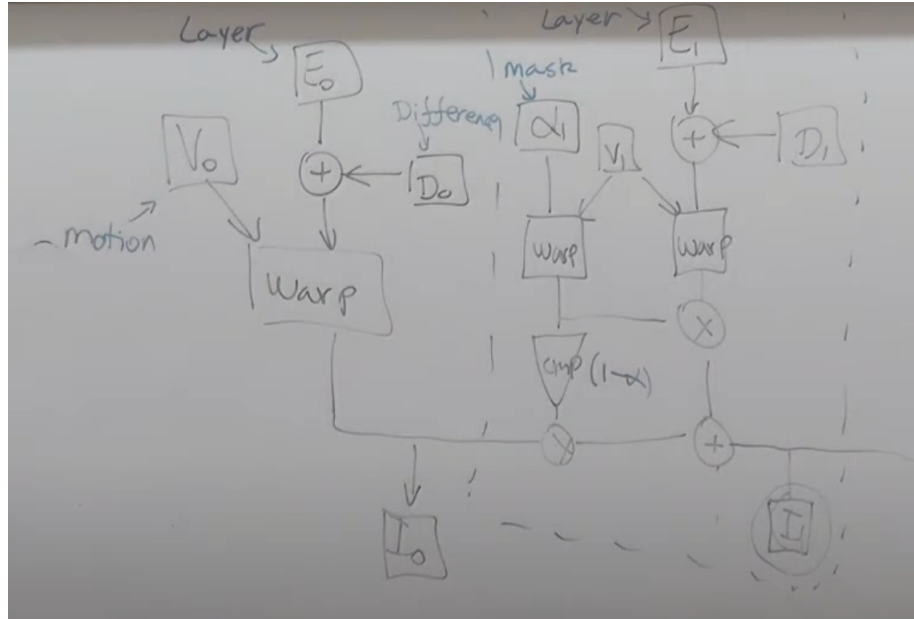
E_k - are the layers (where I_0 is the background layer).

I_k - are the images.

α_k - are the masks.

The second equation tells us that the image I_1 is a combination of the previous image and the current layer. This is done using the mask α_1 (for example, a binary mask), which determines whether we take the pixel from the current layer or from the previous image. The order of the images determines which image overlaps the other.

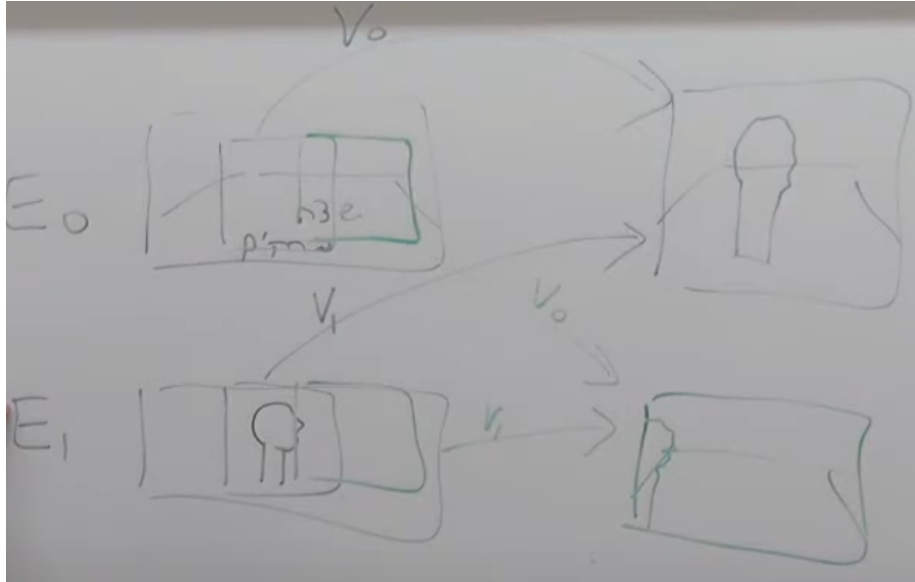
The full synthesis process they came up with is the following:



This diagram illustrates the equations described earlier, where the dashed block is duplicated according to the number of layers. The assumption is that when we observe I_1 , it was generated by the two layers E_0 and E_1 .

To generate I_0 , we start with a background layer E_0 , warp that layer using a vector field V_0 that describes the motion, optionally add a differential image D_0 if needed, and obtain I_0 . The same process is applied to the next layer E_1 ; however, in this case, we also warp with respect to its mask α_1 and then combine the previous image with the current layer using the mask to obtain I_1 .

We will use the following illustration to explain the algorithm:

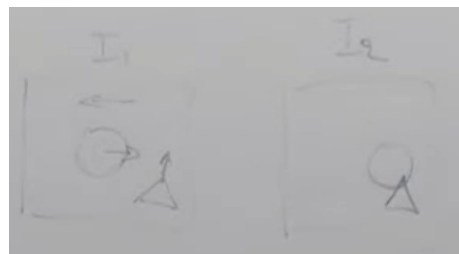


We have two consecutive frames - blue and green. V_0 and V_1 are the motions and change for every timestep. The layers E_0 and E_1 , however, remain constant on every timestep.

What we have shown so far is the synthesis process. What we want, however, is the analysis process: given I_k and the number of layers, we want to determine, for each pixel, which layer it belongs to. In other words, we aim to reverse the process to recover the layers E_0 and E_1 , while simultaneously estimating the motion fields V_k and the masks α_k .

Defining the problem: Given a pair of frames I_1 and I_2 , and the number of layers k , find the segmentation of the motion between the pair of images to k Affine motions.

For example, if we have the following set up:



We want to estimate the motion of all the objects and the motion of the background itself.

Algorithm:

1. L-K on the regions.
2. K-means.
3. Go to 1 until convergence.

We have no a-priori knowledge of which regions move in which direction, so we first run LK on different regions of the image using observation windows. We can use any motion transformation, but for now, let's assume an affine transform. For each window, we obtain a vector of parameters $V \in R^6$, which maps and transforms the window in image I_1 to its corresponding position in image I_2 .

Next, we run K-means clustering in the space R^6 to identify 2-D affine motions that are consistent across the image plane. By clustering windows with similar motion, we can recover the original layers. This process is repeated over several iterations until the algorithm converges to a stable solution. At each iteration, we select **random** windows to perform the pixel-level classification. The algorithm will naturally cluster the most dominant motions in the image.

K-means

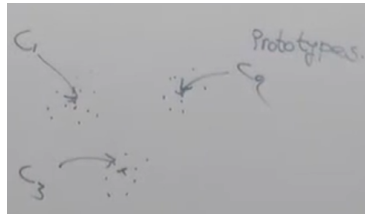
Given a set of points $x \in R_d$, separate them into groups of k . We will define:

$$Encode : E^d \rightarrow [1, \dots, k]$$

$$Decode : [1, \dots, k] \rightarrow E^d$$

$$Distortion = \sum_{i=1}^n \|x_i - Decode(Encode(x_i))\|^2$$

Our goal is to minimize the distortion. For example, if we have the following set of points, we want to find their centers:



We will denote those centers (sometimes called prototypes) as:

$$C_j = Decode[j]$$

and we got:

$$Distortion = \sum_{i=1}^n \|x_i - C_{Encode(x_i)}\|^2$$

Our goal is given dataset x and the number of clusters k , we want to find the prototypes that minimize the distortion.

Algorithm K-Means:

1. Initialize $c_1, \dots, c_k$
2. Assign x_i to the closest c_j
3. $c_j = \frac{1}{|s_j|} \sum_{i \in s_j} x_i$
4. Return to section 2.

This is an iterative algorithm. We first initialize the prototypes randomly. Then, for each data point, we assign the closest prototype. Next, we update the prototypes as the average of all data points assigned to the same prototype. This process is repeated until convergence.

We assigned each datapoint x_i to its closest prototype c_j because it will have the smallest euclidean distance:

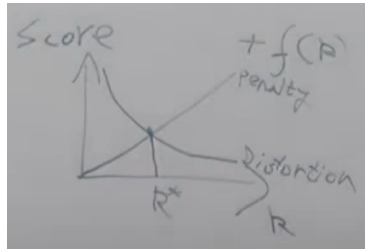
$$\|x_i - c_l\|^2 > \|x_i - c_j\|^2$$

This assignment will minimize the distortion. After that, we take the average and this is because if we differentiate the distortion and compare to 0:

$$\begin{aligned} \frac{\partial Distortion}{\partial c_j} &= \frac{\partial}{\partial c_j} \sum_{i=1}^n \|x_i - C_j\|^2 \\ &= -2 \sum_{i \in s_j} \|x_i - c_j\| = 0 \\ &\longrightarrow c_j = \frac{1}{|s_j|} \sum_{i \in s_j} x_i \end{aligned}$$

Thus, c_j minimize distortion in square error model. Notice after step 2 we right away go to step 3 and won't apply step 2 a second or a third time. That is because after we do the second step we are already optimal so there is no need to do it twice.

Notice that the larger the value of k , the smaller the distortion will be. Ideally, if k equals the number of pixels, the distortion would be zero. There are algorithms based on the *Minimum Description Length* (MDL) principle that aim to find the optimal value of k for clustering:



However, this is out of the scope for this course so we won't dive deeper into it.

7 Video Magnification

Following the previous section, motion magnification (also called Lagrangian Video Processing) estimates the motion vector of each pixel, clusters the pixels into layers, and amplifies the motion in a specific layer.

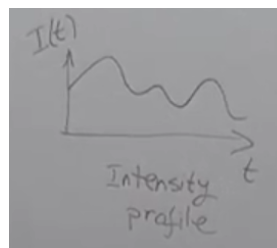
However, a new approach, called Eulerian Video Processing, addresses the same problem in a simpler and more elegant way. The main difference between the two approaches lies in the coordinates: Lagrangian methods track individual pixels over time, whereas Eulerian methods fixate on a region in the frame and observe its intensity profile. In other words, Lagrangian methods operate in the image plane, while Eulerian methods operate in the time domain. In the Eulerian approach, each pixel is observed independently, without considering its neighborhood. The most distinctive property of the Eulerian method is its ability to capture subtle, minor changes.

Given $I(x, t)$ a 1-D image, where $I(x, t)$ is the value of the pixel at time t . We will define:

$$I(x, t) = f(x + \delta(t))$$

$$I(x, 0) = f(x)$$

Intuitively, at every pixel we observe:



Our goal is to calculate:

$$\hat{I}(x, t) = f(x + (1 + \alpha)\delta(t))$$

Here, α is a given amplification factor. There are two approaches to motion magnification. The first is the Lagrangian approach, which operates in the image plane. The second is the Eulerian approach, which observes the temporal derivative. This method links the rate of change over time to motion in the image plane: if a pixel moves quickly in the image plane, its temporal derivative will be large.

For that, we will use first order Taylor approximation:

$$I(x, t) \approx f(x) + \underbrace{\delta(t) \frac{\partial f(x)}{\partial x}}_{B(x, t)}$$

We want:

$$\tilde{I}(x, t) = I(x, t) + \alpha B(x, t)$$

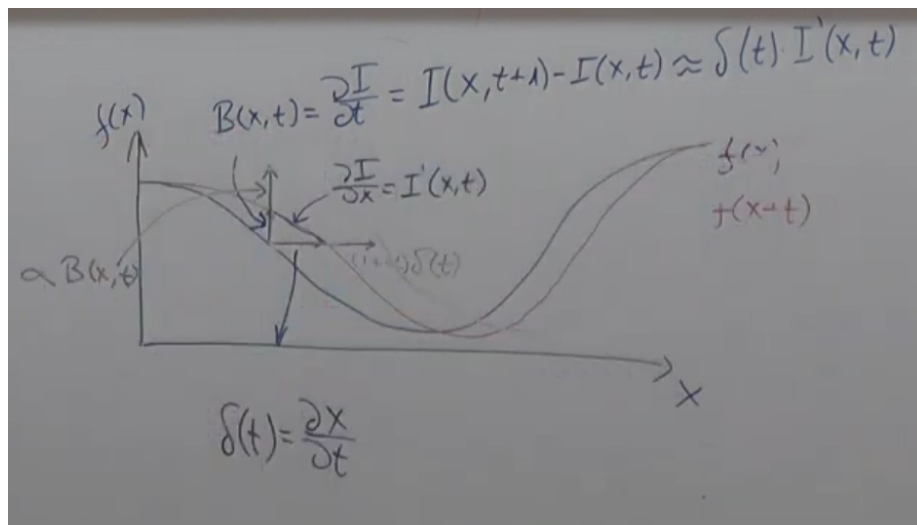
We insert:

$$\tilde{I}(x, t) \approx f(x) + (1 + \alpha)\delta(t) \frac{\partial f(x)}{\partial x}$$

And get:

$$\tilde{I}(x, t) \approx f(x + (1 + \alpha)\delta(t))$$

In practice, we take the difference in time between two frames, amplify it and add to the base image. The following diagram visualizes this:

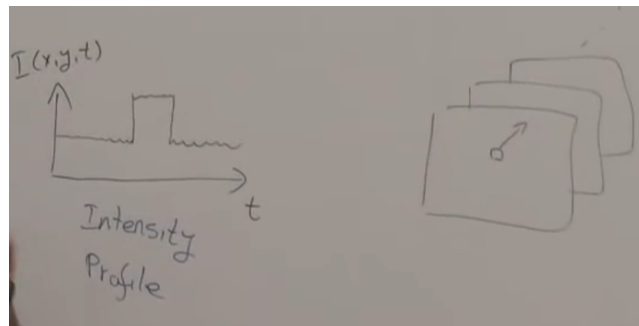


[it is supposed to be $(1 + \alpha)B$ and not αB]

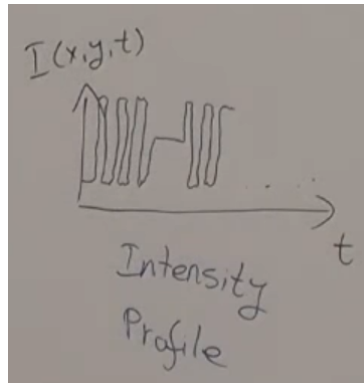
8 Background Substruction

Background subtraction is a fundamental task in video processing. Consider the example of a surveillance camera, where the goal is to detect moving objects. The simplest solution is to compute the difference between the current frame and the previous frame, and mark pixels as moving if the difference exceeds a threshold. Another approach is to compute a weighted average over time and compare the current frame to this background model instead of the immediate previous frame. However, these simple methods have limitations. In this section, we will present the most standard algorithm for detecting sudden moving objects.

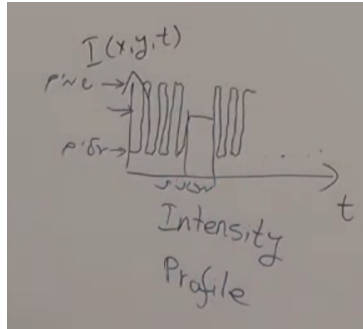
Given a video, we will work on each pixel separately. We will observe its intensity profile over time:



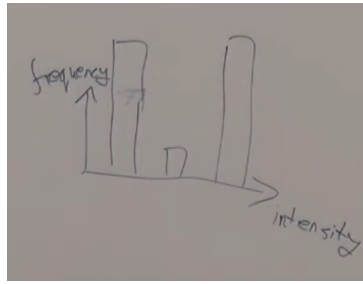
We would expect that initially, a pixel reflects the color of the background. When a moving object enters the frame, the pixel's color changes accordingly. Finally, when the object leaves the frame, the pixel returns to the background color. In practice, however, several phenomena can occur that deviate from this ideal behavior:



A pixel often observes multiple background elements. For example, it may capture moving leaves in the wind. To summarize:



Therefore, the solution propose to do clustering. We would take all the pixel values at the intensity profile and observe their histogram:



In this case, clustering will divide the pixel values into three groups: the two clusters with the highest frequency correspond to the leaves and the sky (the background), while the smaller cluster represents the moving object. Note that the clusters are typically Gaussian-shaped rather than rectangular due to sensor noise.

However, this approach has three main disadvantages. First, the algorithm introduces a delay: it requires collecting many frames for clustering, and by the time the data is stored and the computation is performed, the environment may have already changed. In addition, the per-pixel clustering algorithm is computationally expensive, which further contributes to this delay. Second, the method requires substantial memory to store the frames.

As a result, another approach overcomes these disadvantages by using a dynamic Gaussian Mixture Model (GMM). We consider the intensity of each pixel as a random variable x . Let us start with the distribution of x given the k -th Gaussian component:

$$f_{x|k}(x|k, \theta_k) = \frac{1}{2\pi^d |\Sigma_k|^{d/2}} e^{-\frac{1}{2}(x-\mu_k)^T \Sigma_k^{-1} (x-\mu_k)}$$

Where d is the dimension, x could be in grayscale or in RGB, k is the index of gaussian and $\theta_k = (\mu_k, \Sigma_k)$ are the parameters of gaussian 'k'.

We will use the assumption that the covariance matrix Σ_k is diagonal or is equal to σI for simplicity. The full distribution function (given the set of all the gaussians) would be:

$$f_x(x|\Phi) = \sum_{k=1}^K p(k) f_{x|k}(x|k, \theta_k)$$

Where we denote:

$$\omega_k = p(k), \sum_{k=1}^K \omega_k = 1$$

$$\Phi = (\omega_1, \dots, \omega_K, \theta_1, \dots, \theta_K)$$

$$\theta_k = (\mu_k, \sigma_k)$$

This is similar to the k-means algorithm, where we have the intensity values and we need to answer which pixels belong to which cluster and it is solved iteratively.

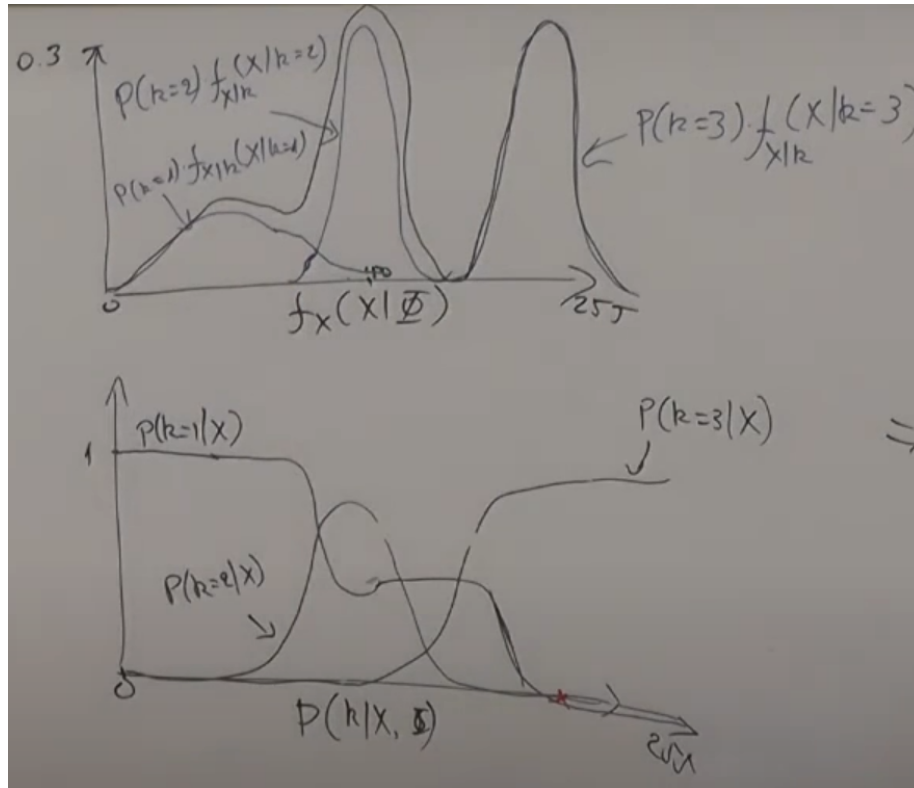
When we get a new frame, for every pixel we want to make the decision in that frame whether it is a background or a moving object. This is done by doing clustering with GMM and the decision will be taken by the gaussian with highest probability. For that, we will use Bayes:

$$p(k|x, \Phi) = \frac{p(k) p_{x|k}(x|k, \theta_k)}{f_x(x|\Phi)}$$

$$\implies MAP : \hat{k} = \arg \max_k P(k|x, \Phi) = \arg \max_k \omega_k f_{x|k}(x|k, \theta_k)$$

As in previous lectures, $p(k)$ is the prior and $p_{x|k}(x|k, \theta_k)$ is the likelihood. The prior in this case tells how much the pixel have seen background objects (for example blue values of the sky or green values of the leaves). The likelihood tells how good the gaussian 'k' (for example the gaussian of the sky or the gaussian of the leaves) describes the pixel.

We will use the following diagram as an example:



We will start the explanation from the upper plot. We have 3 gaussians (marked in blue). The GMM is the sum of all those gaussians (marked in back). For example, the middle gaussian tells pixel's value is 150 with a probability of 0.3.

The bottom plot describes the posteriors of each gaussian. Given the pixel value, we will make the decision by taking the gaussian with the highest posterior. For example, if the pixel's value is 0 we will take gaussian 1, for 255 we will take gaussian 3, etc.

What we are left now to do is to determine which of the gaussians belong to the background and which to the moving object(s). More formally: given GMM, how do we classify the gaussians to background/object?

This is done by first sorting the Gaussians in descending order of $\frac{\omega_k}{\sigma_k}$. Here, ω_k represents how frequently the k -th Gaussian has been observed in the past — the higher its value, the more likely it belongs to the background. σ_k represents the width of the Gaussian; smaller values indicate less variation in intensity, suggesting a static object.

After the sort, we define all the gaussians satisfying the following constraint to be the background:

$$B = \arg \min_b \left(\sum_{k=1}^b \frac{\omega_k}{\sigma_k} > T \right)$$

Where T is user specific threshold.

The full algorithm

The full equation we wish to solve if we have enough resources is:

$$P(x_1, \dots, x_N, k | \Phi) = \prod_{t=1}^N w_k f_{x|k}(x_t | k, \theta_k)$$

If we solve using EM algorithm (won't be discussed in the course) we will get:

$$\begin{aligned} \hat{\omega}_k &= \frac{1}{N} \sum_{i=1}^N p(k | x_t, \Phi) \\ \hat{\mu}_k &= \frac{\sum_{i=1}^N x_t p(k | x_t, \Phi)}{\sum_{i=1}^N p(k | x_t, \Phi)} \\ \hat{\sigma}_k^2 &= \frac{\sum_{i=1}^N ((x_t - \mu_k)(x_t - \mu_k)^T) p(k | x_t, \Phi)}{\sum_{i=1}^N p(k | x_t, \Phi)} \end{aligned}$$

We can't run the full EM algorithm in real time because we can't stop at time t and make the decision. Thus, given GMM and a new measurement, do:

$$match : M_{k,t} = \begin{cases} 1 & \frac{x_t - \mu_{k,t-1}}{\sigma_{k,t-1}} < 2.5 \\ 0 & O.W. \end{cases}$$

$$\omega_{k,t} = (1 - \alpha)\omega_{k,t-1} + \alpha M_{k,t}$$

$$Renormilize : (\omega_{k,t})_{k=1}^K$$

$$\mu_t = (1 - \rho)M_{t-1} + \rho x_t$$

$$\sigma_t^2 = (1 - \rho)\sigma_{t-1}^2 + \rho(x_t - M_t)(x_t - M_t)^T$$

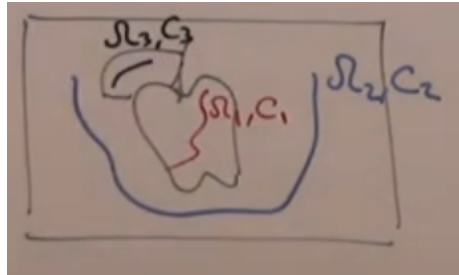
$$\rho = \frac{\alpha \cdot p(k | x_t, \Phi)}{\hat{w}_{k,t}}$$

Instead of calculating $f_{x|k}(x | k, \theta_k)$ the calculate the match. They assumed only one gaussian matches pixel. The first gaussian to match is the first to describe the pixel. Then, we update the weights according to the match.

9 Video colorization

Inputs:

- luminance channel: $Y : \Omega \times [0, T] \rightarrow R$, where $\Omega = [0, W] \times [0, H] \in R^2$ is the parametrization plane of the image or the image domain, and $[0, T]$ is a temporal time interval. If $T=0$, then we have a still image, and a video otherwise. Even though the function maps to R , in practice it will map to $[0, 255]$. This would be the monochrome (luma) video.
- A set of "scribbles": $\Omega_k \in \Omega \times [0, T]$ and their corresponding color c_k - color in some colorspace



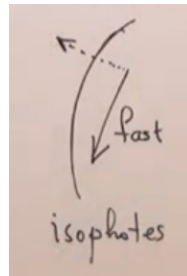
Outputs: We want to create two chromatic channels - $C : \Omega \times [0, T] \rightarrow R^2$ s.t. $C|_{\Omega_k} = c_k$ (we need to be consistent with the input).

Assumption:

- The structural information of chromatic channels can often be inferred from the luminance channel. However, this assumption does not always hold. For example, in the case of fire, the luminance channel may saturate at the maximum value (255) across all pixels, while the chromatic channels still exhibit variation, particularly along object boundaries.
- We will assume region-wise smoothness, modeling the image as a collection of color regions. Within each region, the color is approximately constant, while transitions between regions are sharp—similar to a cartoon image.

Particular case: $T=0$ (image).

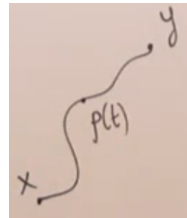
We will do color propagation from (Ω_k) to entire Ω . This scenario considers, for example, a fire that originates at the center of a bush and propagates outward toward the edges of the image. We aim to ensure that the propagation process preserves the underlying image structure. In other words, propagation should occur rapidly within homogeneous regions, but slow down significantly when crossing object boundaries.



We will define a **path** (curvature):

$$(p : [0, 1] \rightarrow \Omega$$

Where $p(0) = x \in \Omega$ and $p(1) = y \in \Omega$:



Now we want to define the **length of the path**:

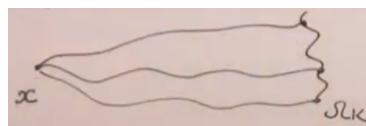
$$L(p) = \int_p | \langle \dot{p}(s), \nabla y \rangle | dl(s)$$

Note that this definition does not correspond to a true geometric length, but rather to a weighting, where the cost increases the more the path follows image contours. The direction that crosses the contours corresponds to the gradient, while the perpendicular direction is the tangent. The cost is higher when the path aligns with the gradient direction. In the particular case where the path remains entirely within a homogeneous region, the cost is zero.

Now that we have defined a length, we can now define a distance, or a geometric distance. In our case, the **distance** would be the minimum length of all the possible paths between any two points:

$$d_L(x, y) = \min_{p: [0, 1] \rightarrow \Omega} L(p)$$

Now we want to define the **point-to-set distance** (or the distance between a point and a region):

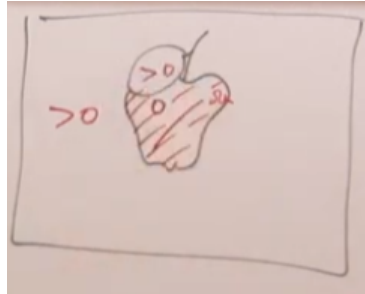


$$d_L(x, \Omega_k) = \min_{y \in \Omega_k} d_L(x, y)$$

We have defined them to build a **distance map**:

$$D_k(x) = d_L(x, \Omega_k)$$

For example, if we have the following image:



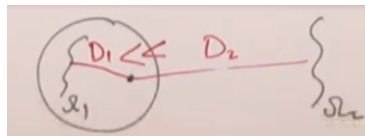
If we consider the apple region and denote it by k , then in the corresponding distance map, the value is zero inside the apple and increases as points move farther away from it. This distance map can be used as an indicator of region membership for each point, or as a weighting function.

Now will define the **blending**:

$$c(x) = \frac{\sum_k W(D_k(x))c_k}{\sum_k W(D_k(x))}$$

This represents a convex combination of the color values, where the weights depend on the distance. For example, the weighting function can be an exponentially decaying function. The choice of this function controls the smoothness of the transition: a sharper decay results in less smooth transitions.

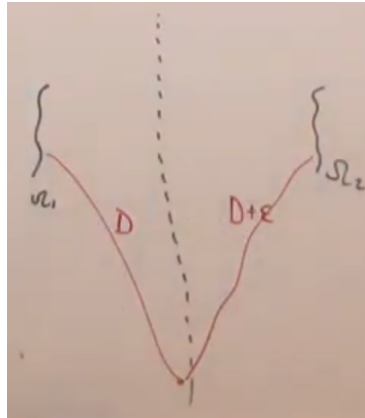
Consider the following example:



We wish to have:

$$\frac{W(D_2)}{W(D_1)} \rightarrow 0$$

In another example, if the point is very far from two scribbles:



The dashed line is considered equi-distance (each point is at the same distance from the two scribbles). In a point closer to one of the scribbles, we wish in the case to have:

$$\frac{W(D + \epsilon)}{W(D)} \rightarrow 0$$

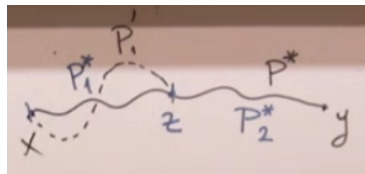
For $D \rightarrow \infty$. In the paper they have proposed the following function:

$$W(D) = \frac{1}{D^b}$$

$$b \in [1, 6]$$

Here, b controls the smoothness of the result, with smaller values producing smoother transitions. The key components we need to compute are $D_k(x)$, and it is important that this calculation is efficient. The remaining question is: how do we compute $D_k(x)$ effectively? We will do that by using Dynamic Programming.

let's observe a path from x to y :



we will assume we have an optimal path P^* with the shortest distance. By observing a middle point z , we can say that P_1^* and P_2^* are the optimal paths of xz and zy respectively. If that wasn't the case, then we would have had another path P_1' shorter than P_1^* , resulting in:

$$L(P_1') < L(P_1^*)$$

$$\implies L(P_1', P_2^*) < L(P^*)$$

Which is a contradiction. This is called **Bellman principle**.

Suppose we want to find a path between x and y . We can do that by observing the neighbors of x and expect that:

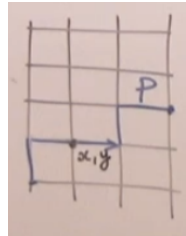
$$\exists z \in N_1(x) :$$

$$d_L(x, y) = d_L(x, z) + d_L(z, y) = L(x, z) + d_L(z, y)$$

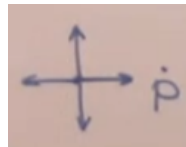


9.1 Dijkstra

We will define a cartesian grid on the image. Suppose we define the following path:



The change in the path can be one of the following options:



On this grid we will also have the gradient of the image, which will be:

$$\nabla Y(x, y) = \begin{pmatrix} Y(x+1, y) - Y(x, y) \\ Y(x, y+1) - Y(x, y) \end{pmatrix}$$

For each edge, we can define a weight. For example, we can take an inspiration from the gradient and assign

$$W = |Y(x+1, y) - Y(x, y)|$$

When we move horizontally, and

$$W = |Y(x, y+1) - Y(x, y)|$$

when we move vertically. Or more generally:

$$e = ((x, y), (x', y'))$$

$$W_e = |Y(x, y) - Y(x', y')|$$

And the length of the path will be:

$$L_p = \sum_{e \in P} W_e$$

$$d_L(x, y) = \min L(p)$$

Where p starts from x and ends at y. Now we will define the algorithm itself.

Input:

- Weighted undirected graph (Ω, E) , $W : E \rightarrow R_+$
- $\Omega_s \in \Omega$ Source set

Output:

$$D : \Omega \rightarrow R_+$$

$$D(x) = d_L(x, \Omega_s)$$

$$D|_{\Omega_s} = 0$$

Dijkstra's algorithm

Initialize:

$$D|_{\Omega_s} = 0$$

$$D|_{\Omega \setminus \Omega_s} = \infty$$

$$Q = \Omega$$

Where Q is a queue.

Iterations: While $Q \neq \emptyset$ do:

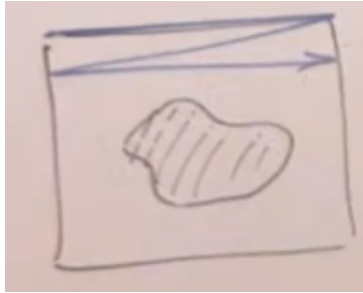
1. $x = \arg \min_{x \in Q} D(x)$
2. Update:
 $\forall y \in N_1(x) \cap Q$
 $D(y) \leftarrow \min\{D(y), D(x) + W(x, y)\}$
3. $Q \leftarrow Q \setminus \{x\}$

Now we will determine the complexity of the algorithm. We start by first observing the number of iterations. Since this algorithm does a single pass over all the pixels, the complexity of the loop will be $O(|\Omega|) = N$. Steps 2 and 3 are $O(1)$, while step 1 will be $O(N^2)$ if we do this naively. However, there's a data structure that manages to make the complexity of step 1 to become $O(\log(N))$. In summary, the complexity is $O(N \log(N))$, which is great.

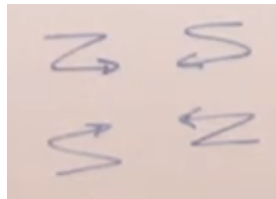
However, in modern days arithmetics are considered very fast. The main bottleneck is the memory itself.

9.2 Danielsson (Fast Sweeping)

He proposed to do a raster sweep:

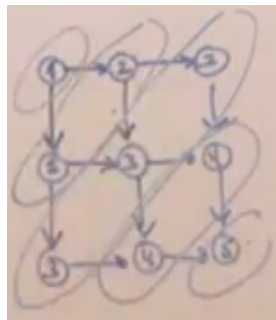


However, this does not give the full distance map, since the sweep is casual and we only observe pixels from the right and then bottom. We thus need to do 4 sweeps:

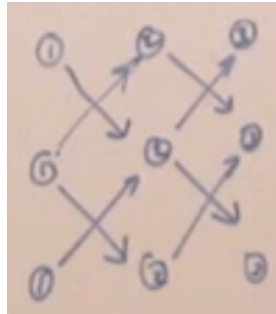


In our case where the distances are not euclidean, we may have paths that form spirals for example. As a result, we may need more than 4 sweeps. If the spiral turns N times, we will need at least $4N$ sweeps. In practice, since we don't know the amount of turns, we'll do sweeps until the distance map doesn't change much. Despite all this, it is a very fast process, faster than using Dijkstra.

Now we will add parallel computation to this. In the following example, we will have 5 steps, where each nodes encircled we can compute in parallel:



However, this is not a good approach since in step 1 and 5 we don't really compute in parallel. A better approach would be changing the neighborhood in the following way:



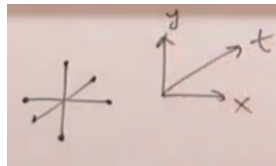
Here we can compute each column in parallel.

9.3 Colorization in video

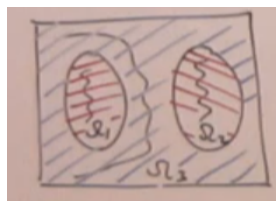
Now the gradient will be:

$$\nabla Y = \begin{pmatrix} \partial_x Y \\ \partial_y Y \\ \partial_t Y \end{pmatrix}$$

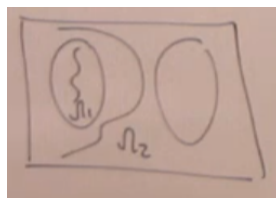
In addition, the neighborhood will have a size of 6 (instead of size of 4):



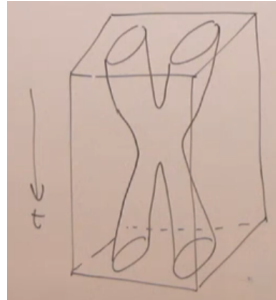
Colorization in video can sometimes simplify the algorithm. Suppose we have two objects coming towards each other, merge and then separate again. If we work only with images, we will need to create 3 scribbles the following way:



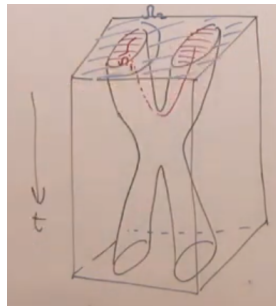
We cannot do 2 scribbles. For example, in the following figure:



Then the right object will be colored with as the 2nd scribble. In video, since we have the dimension of time, the diagram will become 3 dimensional:



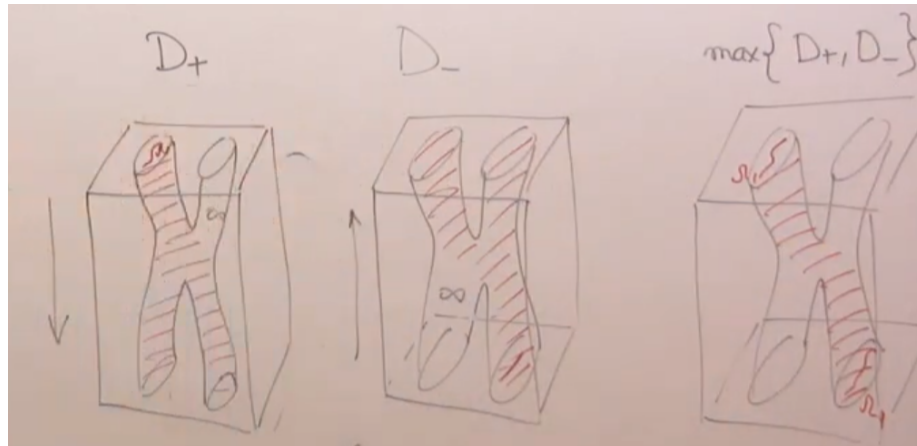
As a result, we can color both objects using only 2 scribbles the following way:



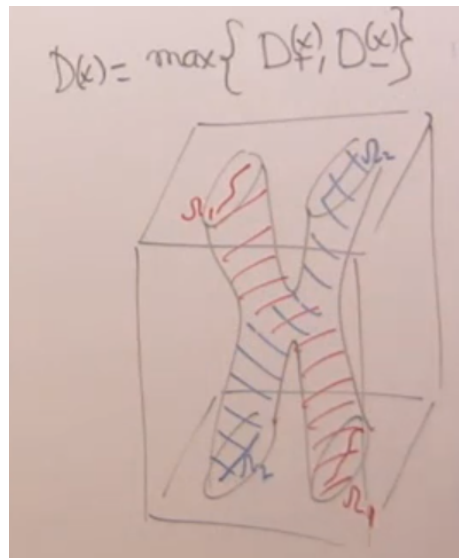
In case we want to avoid this phenomena, we can simply make the t -axis one-directional instead of bi-directional. This can be implemented by making edges in the backward direction have large weights (infinity), where edges in direction of time we keep as previously. However, this won't give the correct result in the future frames:



To fix this, we will need to create two different distance maps (each with different direction on the t-axis), and then apply maximum:



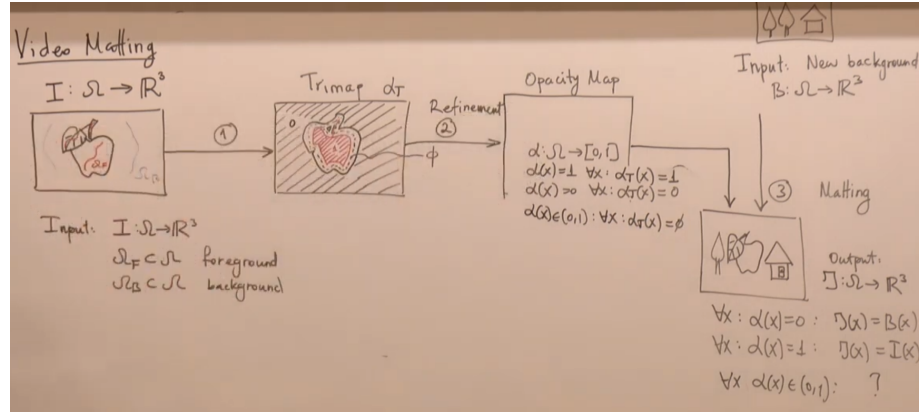
Notice that we don't need to create scribble at every frame, making the work much easier. If we want to color both objects, we will have the following result:



However, this is not a major problem, since in the frames at the middle we will come back and make scribbles exclusively for them.

10 Video Matting

The purpose of video matting is taking the foreground of an image and insert it to a new background. The entire framework of video matting comprises:



Input: A 3-channel image ($I: \Omega \rightarrow \mathbb{R}^3$ or denoted as $c: \Omega \rightarrow \mathbb{R}^3$) and scribbles of the foreground ($\Omega_F \in \Omega$) and the background ($\Omega_B \in \Omega$). We can even add more information to the input (such as optical flow for better segmentation), so the input image does not necessarily belong to $\mathbb{R}^3$.

Trimap: This is essentially a map of only 3 values (hence the name): 1 - we confidently know the pixel is foreground. 0 - we confidently know the pixel is background. Undecided value ϕ - we are not sure if the pixel is foreground or background.

Opacity map: It will fill the undecided region with values which are between 0 and 1.

Matting: This process takes as input an opacity map and a new background, and performs the final matting between the foreground of the original image and the new background. In regions where the opacity map is 0, the new background is inserted, while in regions where the opacity map is 1, the original foreground is preserved.

The question remains on what to insert to pixels where the opacity map was between 0 and 1. One possible way is to implement $\alpha I + (1 - \alpha B)$, but we won't do that. The reason is because in all those semi-transparent areas we will have a mix between the colors of the foreground and the background, which is not desired. For example, if we had a light hair on a red background, the color of the hair will be mixed with red.

10.1 Trimap Estimation:

If we had a light foreground and dark background, the most simple approach would be creating a histogram, dividing the values to bins and finding a threshold for which bins belong to the background. However, we are working with at least 3-channel images, so making for example 10 bins in each dimension, we will have in total 1000 bins which is a lot.

We will, therefore, work with what is called **Kernel Density Estimation (KDE)**. This is a method to estimate a distribution from a finite sample set. What we have is the color value $c \in R^3$ for every pixel, and we want to find the color distribution of foreground and background. The scribbles themselves indicate we have conditional probability: we have a color given the scribble is foreground or background. More formally: $\forall x \in \Omega_F : c(x) \in f_{C|F}$ and $\forall x \in \Omega_B : c(x) \in f_{C|B}$.

We will start with an example in 1-D. Usually, if we want to create a histogram, we would divide the axis to bins and count how many pixels we had at every bin. Instead, we would draw around each pixel a "bell".



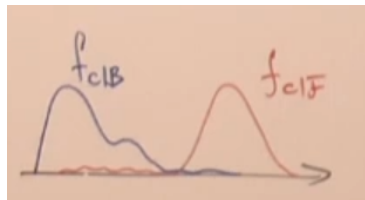
We estimate:

$$f(c|F) \propto \sum_{x \in \Omega_F} K(c, c(x))$$

The same will be done with the background. This is sometimes called Parzen window. Usually the function K is:

$$K(c, c') \propto e^{-\frac{\|c - c'\|_2^2}{2\sigma^2}}$$

An ideal histogram would look like:



However, in most cases, we will have a very large overlap. The complexity would be for calculating $f(c|F)$ is $O(|\Omega| \cdot |\Omega_F|)$ since K gets two parameters c and c' . This is too large, and as a result we will do something else. We will assume for now we have calculated them, and later introduce a practical approach for them.

We will develop the equations:

$$P(F|c) = \frac{f(c|F)P(F)}{f(c)} = \frac{f(c|F)P(F)}{f(c|F)P(F) + f(c|B)P(B)}$$

$$P(B|c) = \frac{f(c|B)P(B)}{f(c)} = \frac{f(c|B)P(B)}{f(c|F)P(F) + f(c|B)P(B)}$$

The main parameters unknown to us here are $P(F)$ and $P(B)$. In worst case, we can assume uniform distribution:

$$P(F) = P(B) = \frac{1}{2}$$

$$\implies P(F|c) = \frac{f(c|F)}{f(c|F) + f(c|B)}$$

Now we will do distance maps similar to video colorization. We had a path $p \in \Omega$ but now:

$$L(p) = \int_p | \langle W(p(t)), \dot{p}(t) \rangle | dl$$

$$W(x) = \nabla_x p(F|c(x))$$

We have taken this function because it will be consistently larger in the foreground and smaller in the background. Next:

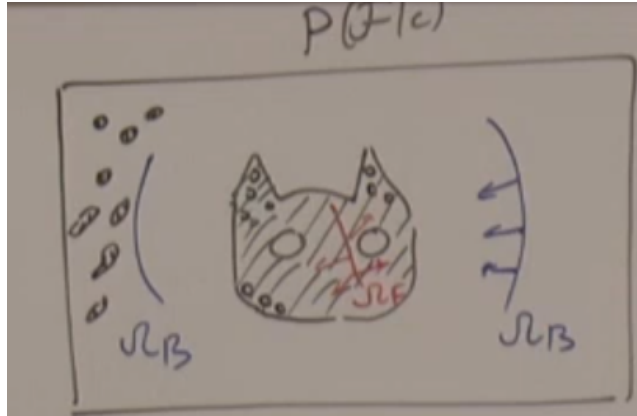
$$x, y \in \Omega : d_L(x, y) = \min_{p \in [x, y]} L(p)$$

Where we go over all the points p from x to y and find the minimum. Next:

$$D_F(x) = d_L(x, \Omega_F) = \min_{y \in \Omega_F} d_L(x, y)$$

$$D_B(x) = d_L(x, \Omega_B) = \min_{y \in \Omega_B} d_L(x, y)$$

This will eventually create two regions: the first region would be for the foreground and the second would be for the background.



Within each region, pixels will have similar distance values, whereas crossing between regions leads to a sharp change in distance. Consequently, in each distance map, one region will exhibit relatively small values while the other will have larger values. By comparing the two distance maps, we can determine whether a given pixel belongs to the foreground or the background:

$$V_F = (x \in \Omega : D_F(x) < D_B(x))$$

$$V_B = (x \in \Omega : D_F(x) > D_B(x))$$

And we can also define the boundary:

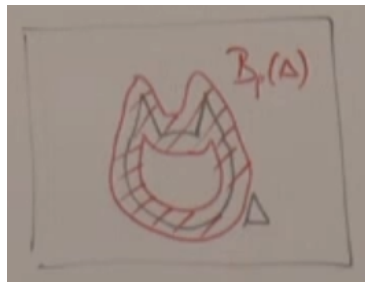
$$\Delta = (x \in \Omega : D_F(x) = D_B(x))$$

Usually, this set will be empty because the probability for the distance to be equal is very low. As a result, the boundary can be found using interpolation. Now, we will create a narrow band around the boundary:

$$B_p(\Delta) = \bigcup_{x \in \Delta} B_\rho(x)$$

Where $B_\rho(x)$ is a ball around x with radius ρ . We can also write this as:

$$x \in B_\rho(\Delta) \iff \exists y \in \Delta : \|x - y\| \leq \rho$$

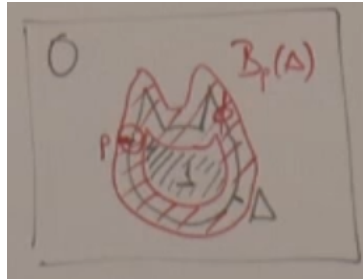


Finally, the Trimap will be:

$$\forall x \in V_F \setminus B_\rho(\Delta) : \alpha_T(x) = 1$$

$$\forall x \in V_B \setminus B_\rho(\Delta) : \alpha_T(x) = 0$$

$$\forall x \in B_\rho(\Delta) : \alpha_T(x) = \phi$$



10.2 Trimap Refinement - Opacity map

Let's draw only the area that is interesting for us:



At this stage, we now have a much richer set of data. Previously, we only had a few scribbles indicating foreground and background, which were the sole sources of information for the entire image, despite its large color variability. Now, our focus is on a specific region with much more limited color variation. We have a lot more samples of the foreground and background on the **boundary**.



Thus, we will estimate again the distribution functions for refinement. And since we have a curve, we can put a coordinate system and measure the length of the curve. We will denote that as τ . Every point we can denote as $(\tau, c(x))$. Since the boundary is very narrow, this problem is nearly 1-D. Now we can develop the equations:

$$\begin{aligned}\forall x \in B_\rho(\Delta) : \\ \alpha() &= \frac{w_F(x)}{x_F(x) + w_B(x)} \\ w_F(x) &= \bar{D}_F(x)^{-r} \bar{P}_F(x) \\ r &\in (0, 2]\end{aligned}$$

Here $\bar{P}_F(x)$ is computed based on previous step. The equation of $w_F(x)$ was chosen intuitively in the paper.

10.3 Matting

Now we are given:

B - new background

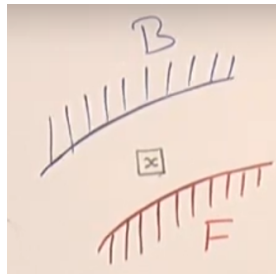
α - opacity map

c - original image.

As said earlier, we could do:

$$J(x) = \alpha c(x) + (1 - \alpha(x))B(x)$$

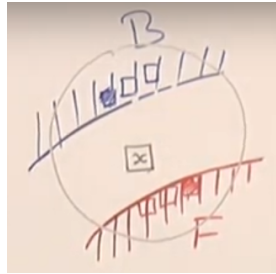
This will be done on the foreground and background, but not on the boundary. That is because the colors will mix and we even may get a halo around the foreground. We will do something else then. Suppose we have the point x:



We can write down the value of the pixel as:

$$c(x) = \alpha f(x) + (1 - \alpha(x))b(x)$$

Where in this case $f(x)$ and $b(x)$ are some unknown foreground and background values respectively. Our goal would be to estimate them. This is an ill-problem (one equation with 2 unknowns), but we have a lot of colors around the pixel that we know they are foreground and background. We will open a window around the pixel, and claim that there are two pixels in the window satisfying the equation above.



On a very small area this assumption most of the time works. What we will do is each time take two pixels in the window, compute the value according to the equation earlier, subtract $c(x)$, calculate the norm and find those which minimize the result:

$$(x_F^*, x_B^*) = \arg \min_{\substack{x_F \in N(x) \cap \{x: \alpha(x)=1\} \\ x_B \in N(x) \cap \{x: \alpha(x)=0\}}} \|\alpha(x)c(x_F) + (1 - \alpha(x))c(x_B) - c(x)\|^2$$

With that we can take the color in x_F^* as the value of the foreground object than $c(x)$:

$$J(x) = \alpha c(x_F^*) + (1 - \alpha(x))B(x)$$

10.4 KDE - Kernel Density Estimation

Now we are only left to calculate KDE efficiently. We want to calculate $\forall x \in \Omega$:

$$f(x) = \sum_{y \in \Omega'} k(x, y)w(y)$$

The computation efficiency here is $O(|\Omega| \cdot |\Omega'|)$. We assume the kernel is low-rank. To understand this, let's assume we have:

$$f_{n \times 1} = K_{n \times m} w_{m \times 1}$$

Low rank means:

$$\text{rank}(K) = p \ll m, n$$

The computation efficiency here is $O(mn)$. We can write this as a sum of matrices with rank 1:

$$K = \sum_{i=1}^p \phi_i \psi_i^T = \begin{pmatrix} | & \dots & | \\ \phi_1 & \dots & \phi_p \\ | & \dots & | \end{pmatrix} \begin{pmatrix} - & \psi_1^T & - \\ \vdots & \vdots & \vdots \\ - & \psi_p^T & - \end{pmatrix} = \Phi_{n \times p} \Psi_{m \times p}^T$$

The rank of this matrix is maximum p . Now we will insert this to previous equation:

$$f = \Phi \Psi^T w$$

The computation time of $\Psi^T w$ is $O(pn)$, where p is very small. If we denote $a = \Psi^T w$, then $f = \Phi a$ will take $O(pn)$. In total we will have $O(p(m+n))$. Instead of quadratic complexity, we have linear complexity. If we say that our kernel $K(x,y)$ is low rank, then:

$$p \ll |\Omega|, |\Omega'|$$

we can write it as:

$$K(x, y) = \sum_{i=1}^p \phi_i(x) \psi_i^T(y)$$

$$a_i = \sum_{y \in \Omega'} \Psi_i(y) w(y)$$

$$f(x) = \sum_{i=1}^p a_i \phi_i(x)$$

The calculation of a_i takes $O(|\Omega'|p)$, the calculation of the last equation also takes $O(|\Omega|p)$ - in total $O((|\Omega| + |\Omega'|)p)$. Thus there is a lot of consideration when choosing the kernel and to pick one which is low rank. Gaussian kernel is for example infinite-rank, but if we take its singular values we will see they decay to 0. We will not discuss this in the course, but the methods for factorizing kernels is called **Fast multiple method**.

11 Re-targeting and Patch Match

11.1 Introduction

The goal of this algorithm is to perform operations on an image—such as resizing, altering the composition, or adding/removing objects—while preserving a natural appearance. For instance, if we naively shrink the image along the x-axis, all objects would appear unnaturally thinner.

The algorithm, however, produces a resized image in which the objects retain their original appearance. The key is that it can selectively remove objects, adjust their positions, or even duplicate them as needed to preserve a natural look.

However, there are cases where the algorithm may fail—for instance, straight lines that become bent or objects that are distorted. To address this, the algorithm can incorporate constraints by manually marking objects whose shape we want to preserve. These constraints can also serve other purposes, such as specifying objects to be removed rather than preserved.

The first algorithm to achieve this was called "Re-targeting", which captured the core idea but was computationally too expensive to be practical. Later, the PatchMatch algorithm overcame this limitation and was able to operate in real time.

11.2 Re-targeting - Formalization

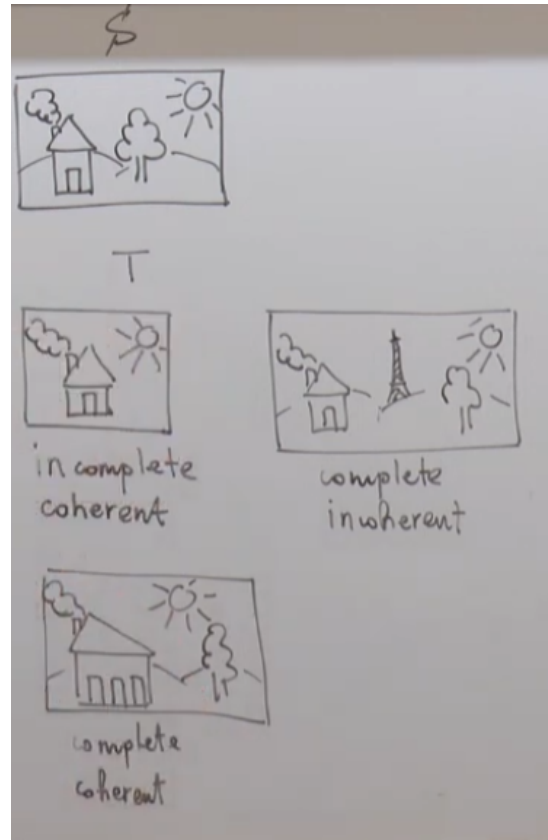
Input:

- **Source:** $S : \Omega \rightarrow R^3$.
 $\Omega = [0, W] \times [0, H]$ - still image
 $\Omega = [0, W] \times [0, H] \times [0, F]$ - video
- **Target domain:** $\Omega' = [0, W'] \times [0, H'] \times [0, F']$ a different pixel matrix which has different resolution, aspect ratio or even different duration if it is a video.

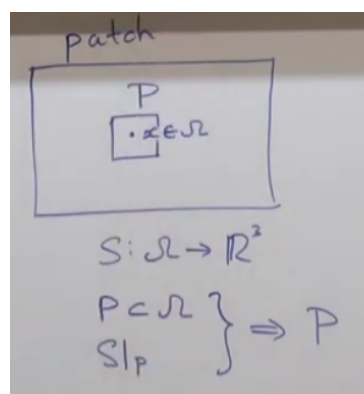
Output: Target $T : \Omega \rightarrow R^3$ with the following constraints:

1. **Completeness:** T contains as much visual information in S as possible. We will later define what is visual information.
2. **Coherence:** T doesn't contain visual information absent in S. For example, if the algorithm adds a window in a building, that window was already in the image but repositioned or duplicated.
3. **Additional constraints:** for instance preserving the object's form, erasing an object, etc.

Some examples describing the constraints:



Now will use some notations:



We will define a **patch** to be a sub-region of the image around some point x . We could denote the patch as $P \in \Omega$ or $S|_P$, but those notations are a bit complex, so we will do abuse of notations and just denote it as P .

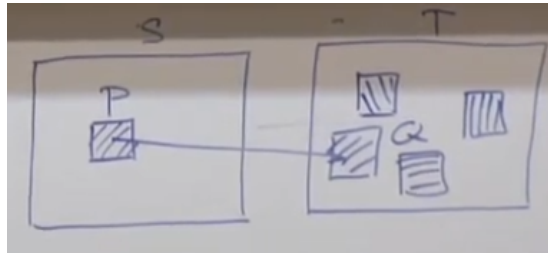
Next, we define a **distance** between two patches P and Q as a dissimilarity measurement between them, and denote it as $d(P, Q)$:

$$d^2(P, Q) = \|P - Q\|_F^2$$

But for simplicity we will use for now:

$$d^2(P, Q) = \|P - Q\|_2^2 = \sum_{i,j} \|P_{ij} - Q_{ij}\|_2^2$$

This distance will be used in order to define coherence and completeness. For that, let us assume we are given a source image and a target image:



We want to find neighboring patches which are similar to patch P. In other words, we want to minimize the distance $d(P, Q)$ to find a patch Q^* which is the most similar:

$$Q^* = \arg \min_{Q \in T} d^2(Q, P)$$

. Using all the notations so far we will be able to define coherence and completeness.

Completeness

The definition of completeness will be:

$$D_{com}(S, T) = \frac{1}{N_s} \sum_{p \in S} \min_{Q \in T} d^2(Q, P)$$

We go over all the patches in S, find its corresponding patch Q in T which gives a minimum distance, and calculate their average. As defined earlier, completeness checks if the target image T contains as much visual information in S as possible. If a patch Q has the same visual information as P, then their distance will be **low**. Thus, we check if all the patches in S have the same visual information as patches in Q by doing the summation of all the minimum distances. The division by N_s will just normalize in order to work with reasonable value range.

Coherence

The definition of coherence will be:

$$D_{coh}(S, T) = \frac{1}{N_T} \sum_{Q \in T} \min_{P \in S} d^2(P, Q)$$

Here we do the opposite: we go over all the patches in the target image and check if there are patches in source image that look different. A patch in T which looks different that all the patches in S will have large minimum distance.

Bidirectional dissimilarity

Now we will use completeness and coherence to define a single loss function:

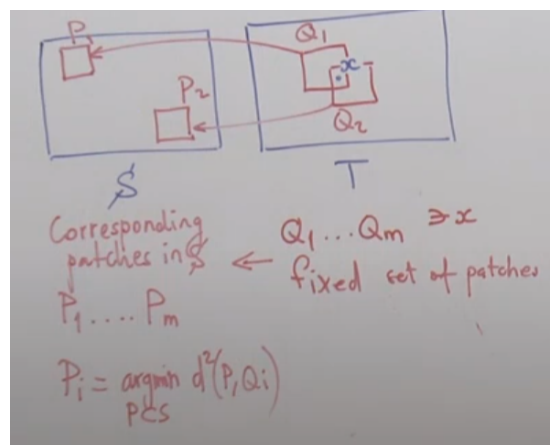
$$D(S, T) = \alpha D_{com}(S, T) + (1 - \alpha) D_{cho}(S, T)$$

Where $\alpha \in [0, 1]$. Thus, image re-targeting can be achieved by:

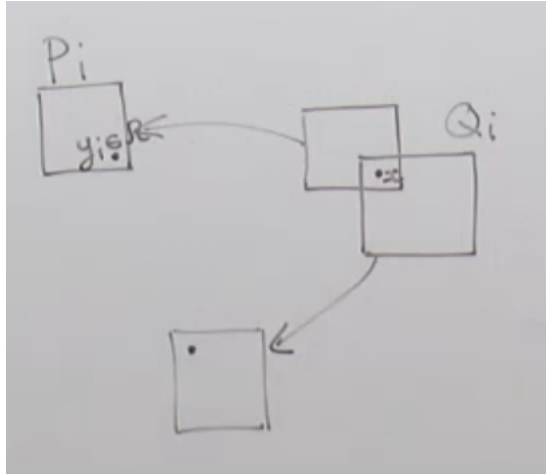
$$T = \arg \min_{T: \Omega' \rightarrow R^3} D(S, T)$$

To solve this, we will each time fix all the pixels except one and ask what should be its value to minimize the loss, and we will do that operation for all the pixels. Formally, we will ask the contribution of $T(x)$ to $D(S, T)$ where $x \in \Omega'$:

1. **D_{coh}** : In this case we go over all the patches in T. However, the only patches which will be relevant with the pixel x are the patches which contain it:



The number of patches is quite known. If we define the patch size and the stepsize between successive patches then it will be straightforward. Each patch Q_i has its own corresponding patch P_i (also called its **Nearest Neighbor**). If we observe more closely:



We can transfer x to the coordinates of the patch (for example the top left corner of the patch). Thus, if P_i is the corresponding patch of Q_i , then the corresponding point of x in patch P_i would be in the same patch coordinates and we will denote that point as y_i . As a result, the contribution of $T(x)$ would be:

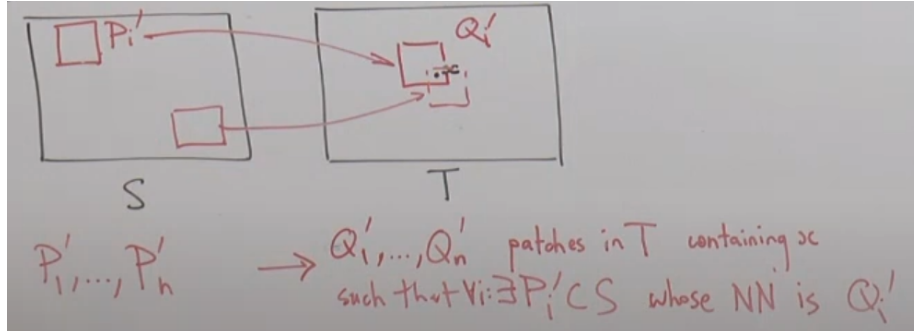
$$\frac{1}{N_T} \sum_{i=1}^m \|S(y_i) - T(x)\|_2^2$$

Where m is the number of patches containing x and it is known. Thus, $D_{coh}(S, T)$ can be re-written as:

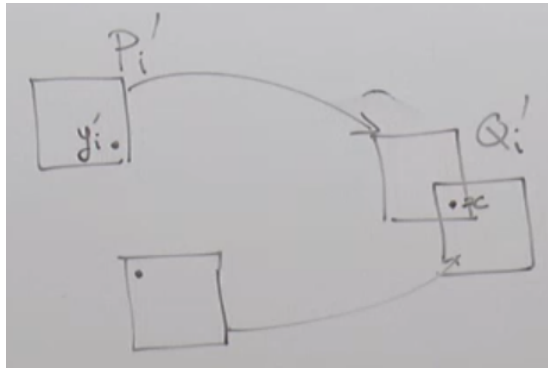
$$D_{coh}(S, T) = \frac{1}{N_T} \sum_{i=1}^m \|S(y_i) - T(x)\|_2^2 + Const$$

Where the constant is the contribution of other pixels we fixed.

2. **D_{com}** : In coherence, the patches in T containing x are well defined and constant. Here, however, it is a bit more complicated. The task would be more simple if we were observing patches in S and searching for the corresponding one to a patch in T containing x . However, we are observing a set of patches in T containing x and take only those who has any corresponding patches in P . The amount of patches here is not constant, since not every patch Q_i will necessary have a corresponding patch P_i . In fact, there could be a point x in T where no patches containing it have any corresponding ones in S .



In practice, we will go over all the patches in T containing x and take only those who have Nearest Neighbor in S . As before, if we observe closely:



This is similar to what we have in coherence. Finally, the contribution would be:

$$\frac{1}{N_S} \sum_{i=1}^n \|S(y'_i) - T(x)\|_2^2$$

Thus, the contribution of x to $D(S, T)$ would be:

$$D(S, T) = \underbrace{\frac{\alpha}{N_S} \sum_{i=1}^n \|S(y'_i) - T(x)\|_2^2 + \frac{1-\alpha}{N_T} \sum_{i=1}^m \|S(y_i) - T(x)\|_2^2}_{E(T(x))} + \text{const}$$

In order to find the minimum, we will differentiate and demand it to be 0:

$$\begin{aligned} \frac{\partial E}{\partial T(x)} &= 2 \left[\frac{\alpha}{N_S} \sum_{i=1}^n (T(x) - S(y'_i)) + \frac{1-\alpha}{N_T} \sum_{i=1}^m (T(x) - S(y_i)) \right] := 0 \\ \left(\frac{\alpha n}{N_S} + \frac{(1-\alpha)m}{N_T} \right) T(x) - \frac{\alpha}{N_S} \sum_{i=1}^n S(y'_i) - \frac{1-\alpha}{N_T} \sum_{i=1}^m S(y_i) &= 0 \end{aligned}$$

$$\Rightarrow T(x) = \frac{\frac{\alpha}{N_S} \sum_{i=1}^n S(y'_i) + \frac{1-\alpha}{N_T} \sum_{i=1}^m S(y_i)}{\frac{\alpha n}{N_S} + \frac{(1-\alpha)m}{N_T}}$$

Intuitively, the minimum of norm 2 would usually be an average, which we got here in some sense. Now we will write this down as an optimization algorithm:

Re-targeting Algorithm

1. Initialization: $T^1 : \Omega' \rightarrow R^3$
2. For $k = 1, 2, \dots$ until convergence:
 - (a) $\forall x$:
 - i. $T^{k+1}(x) = 0$
 - ii. $C(x) = 0$ - an accumulator for finding the denominator
 - (b) For each $Q \in T^K$:
 - i. find $P = \arg \min_{P \in S} \|P - Q\|_2^2$
 - ii. $T^{k+1}|_Q \leftarrow T^{k+1}|_Q + \frac{1-\alpha}{N_T} P$ - In the subset of image T where we have the patch Q we add the contribution of P. Only to the region Q we add this contribution.
 - iii. $C|_Q \leftarrow C|_Q + \frac{1-\alpha}{N_T}$
 - (c) For each $P' \in S$:
 - i. find $Q' = \arg \min_{Q \in T} \|P - Q\|_2^2$
 - ii. $T^{k+1}|_{Q'} \leftarrow T^{k+1}|_{Q'} + \frac{\alpha}{N_S} P'$
 - iii. $C|_{Q'} \leftarrow C|_{Q'} + \frac{\alpha}{N_S}$
 - (d) $T^{k+1} \leftarrow \frac{T^{k+1}}{C}$, where the division of the matrices is element-wise.

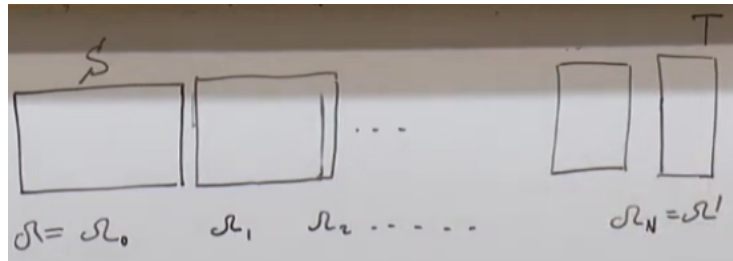
There are a couple of disadvantages to this approach. When we find corresponding patches based on T^k , if in previous iteration we got something legitimate, so we will have here. Thus, everything depends on the initialization of T^1 . If we start with a black image then nothing will work. As a result, we need to make the initialization correctly.

In addition, searching for every patch in S its Nearest Neighbor in T takes a lot of computation time. We will need, therefore, to find a practical solution for this. This will be later be shown with Patch Match.

Initialization of Re-targetting

If S and T have the same (or similar) dimensions, we could pick S as the initialization of T . Otherwise, one way we could initialize is by taking S and crop or resize to the dimensions of T . However, resizing will distort the objects and cropping could potentially miss a lot of visual information.

A better approach would be the following:

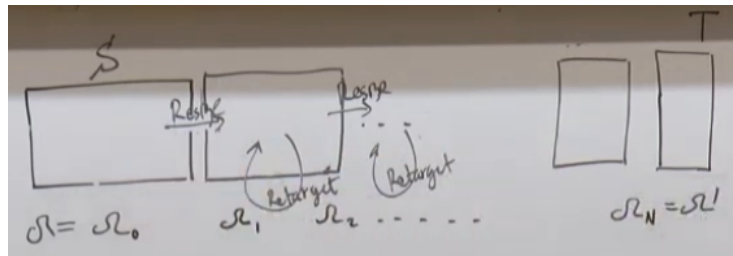


We first convert S from domain Ω_0 to domain Ω_1 which has very similar dimensions to Ω_0 . We then do the same for a couple of hierarchies (where every successive domains are close to each other) until we get $\Omega_N = \Omega'$.

Mathematically, it will be written in the following way:

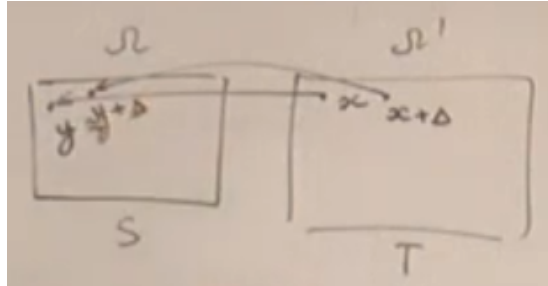
Initialization algorithm:

1. $T_0 = S$
2. For $k=1, \dots, N$:
 - (a) $T_k^{init} = \text{Resize}(T^{k-1}, \Omega_k)$ - the resize itself is very minor because successive domains are nearly the same
 - (b) $T_k = \arg \min_{T: \Omega \rightarrow R^3} D(S, T)$, initialized with T_k^{init} . We refine the resize with re-targeting.



11.3 Patch Match

PatchMatch is designed to solve the Retargeting problem in a reasonable amount of time. The main bottleneck is finding matching patches. PatchMatch assumes spatial coherence: if a patch P in image S matches patch Q in image T , then the neighbors of P are likely to match patches near Q . This means that for a neighboring patch P' , we don't need to search the entire image T , only the area around Q . This greatly reduces the search time and speeds up the computation.

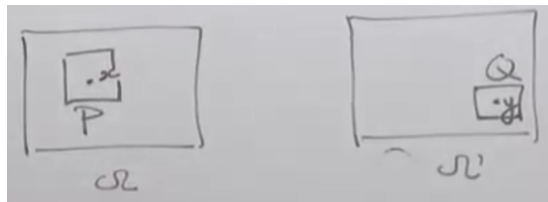


The basic algorithm relies on two key components: local prediction, which exploits spatial coherence, and randomization, which prevents the algorithm from being overly biased by local predictions. Randomization is implemented by examining a window around each patch and randomly selecting candidate patches within that window.

Now we will do this more formally. We are searching:

$$\forall P \in S : Q = \arg \min_{Q \in T} \|P - Q\|$$

Where $S : \Omega \rightarrow R^3$ and $T : \Omega' \rightarrow R^3$.



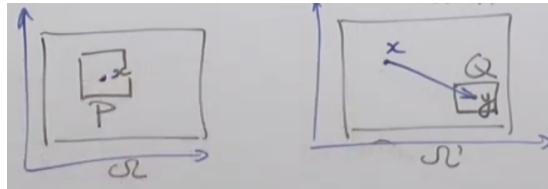
For every x , we have **matching field** f with maps x to its corresponding point y :

$$f : x \in \Omega \rightarrow y \in \Omega'$$

$$f(x) = \arg \min_{y \in \Omega'} \|P_x - Q_y\|_2^2$$

Where patch P is centered around x and patch Q is centered around y .

The coordinate system of Ω and Ω' are considered to be the same, so the position of x is technically valid in Ω' :



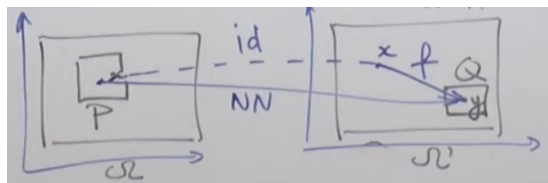
As a result, we can look at their offset instead:

$$f : x \in \Omega \rightarrow y \in \Omega' - x$$

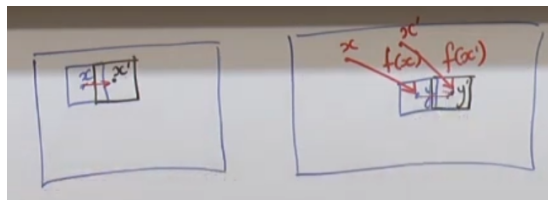
$$f(x) = \arg \min_{y \in \Omega'} \|P_x - Q_y\|_2^2 - x$$

The original mapping can be done by adding the identity:

$$y = x + f(x)$$



We write this down in the following way because the coherence assumption results in the same offset:



In other words, if $f(x)$ is the offset from x to y , then the coherence assumption will make a close point x' to be mapped to y' with offset $f(x')$ which is the **same** as $f(x)$:

$$f(x) \approx f(x')$$

1. Initialization:

2. For $k = 1, 2, \dots$ until convergence

- $$\begin{aligned} f(x) &= \arg \min_{f \in \{f(x), f(x-e_1), f(x-e_2)\}} \|P_x - Q_{x+f}\| \\ d(x) &\leftarrow \min_{f \in \{f(x), f(x-e_1), f(x-e_2)\}} \|P_x - Q_{x+f}\| \end{aligned}$$

[illegible]

-
- A scatter plot with a fitted curve. The curve is labeled $X + f(x) + r$.

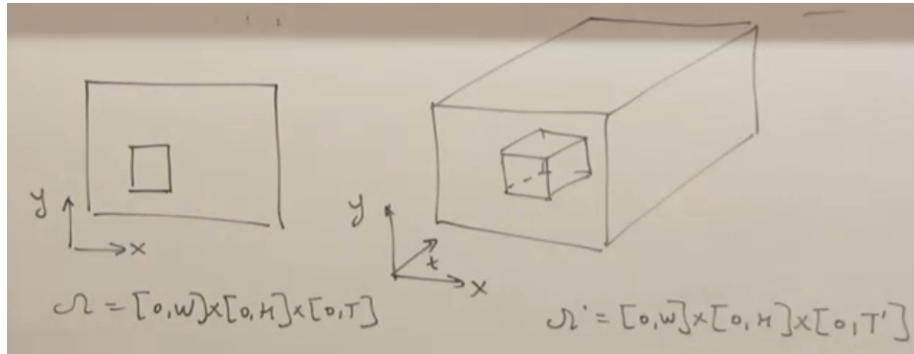
- A. Draw $r \sim \text{Exp}[-w, w]^2$, where w is the scanning window size.
- B. Do:

C. Save:

$$d(x) \leftarrow \min_{f(x), f(x)+r} \|P_x - Q_{x+f}\|$$

11.4 Generalization in Video

This section will focus on how to handle with 3D patches in a video.



A patch in a video captures not only the spatial appearance of an object but also how its visual content changes over time—its temporal dynamics. For objects moving along complex paths, such as circular motion, representing their motion with simple cubic patches can be inefficient and inconvenient.

In video, we have more tasks to handle than we could with images. For example, we can keep the dimensions of the frames while only changing the duration of the video. Same as before, we cannot use scaling in the spatial dimensions because it won't preserve completeness and coherence. That is also true in the time domain - if we naively remove frames (shrink from $[0, T]$ to $[0, T']$) we won't be able to maintain coherence and completeness. However, the equivalents of completeness and coherence are a bit different in the time domain. Maintaining the scale in the time domain would be described by preserving the playback rate (or velocity).

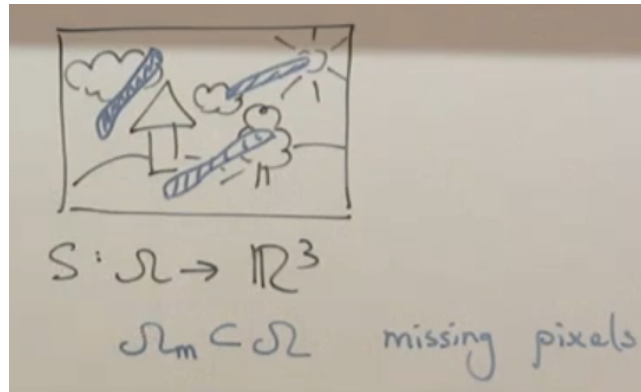
As mentioned earlier, the most naive way to speed up a video is simply to remove frames at regular intervals. While fast, this often makes the scene difficult to follow or understand. PatchMatch offers a more informed approach: it can identify parts of the video that are predictable and safely remove or merge them. By considering patches not only in space but also across time, we can describe the content of a current patch using information from both past and future frames. In this offline setting, temporarily ignoring strict causality is acceptable, as it allows for smoother and more visually coherent results.

Everything we have done in the image domain can also be easily used in videos as is with some generalizations. Of course in this case the time

complexity will increase. We will now go on other examples on the image domain, but they would also be also applicable in video.

11.5 Inpainting

Let's assume we have a source image corrupted by noise which removed some of the pixels:



An example could be removing peoples' faces in photos or an old painting where the paint lost its color in some areas. Our task here is to complete the missing parts. A constraint on the target image T would be that in areas without missing pixels we would expect to have the same pixels as in the source image S . More formally:

Input:

- Source image $S: \Omega \rightarrow \mathbb{R}^3$
- $\Omega_m \in \Omega$ missing pixels

Output: Target image $T: \Omega \rightarrow \mathbb{R}^3$ with the constraint $T|_{\Omega \setminus \Omega_m} = S|_{\Omega \setminus \Omega_m}$.
We again want to find:

$$\min_{T|_{\Omega_m}} D(S, T)$$

s.t $T|_{\Omega \setminus \Omega_m} = S|_{\Omega \setminus \Omega_m}$. This will be done by taking the framework of Patch Match and modify some of the operations.

11.6 Re-shuffling

One application involves taking a region from a source image and pasting it into a target image, similar to reshuffling or rearranging objects within the image. To handle this, we first fix the regions where objects have been pasted, and then ask the algorithm to fill in the remaining areas. We can even combine objects from multiple source images into a single target image; in such cases, PatchMatch is applied using all source images to find suitable matches for the empty regions.

11.7 Additional Constraints

Sometimes it is useful to introduce additional constraints to the task. For example, we may manually mark a face to prevent distortion, mark a straight line to keep it from bending, or mark an area to ensure it is preserved during retargeting. In such cases, we incorporate these constraints by adding weights to $D(S,T)$, assigning higher weights to the constrained regions to prioritize their preservation.

The full equation of D was composed of two summation elements. The first was for patches in image S and the second was for patches in image T . The weighting on the source image is simply done by multiplying with a patch-wise weight W_P (for example larger weight near the center of the image, or some sort of statistics on the pixels inside the patch).

The weighting on the target image is done by using the weighting of the Nearest Neighbor P^* of each patch Q - denoted as W_{P^*} , where:

$$P^* = \arg \min_{P \in S} \|P - Q\|$$

The final equation of $D(S,T)$ will become:

$$D(S,T) = \alpha \frac{\sum_{P \in S} W_P \min_{Q \in T} \|Q - P\|}{\sum_{P \in S} W_P} + (1-\alpha) \frac{\sum_{Q \in T} W_{P^*} \min_{P \in S} \|P - Q\|}{\sum_{Q \in T} W_{P^*}}$$

This is similar to the weighted averaging approach discussed earlier, and we use this formulation because it is convex. If there is a patch whose content we want to preserve—such as a patch containing a face—we assign it a higher weight in the source image, as well as to its nearest neighbor in the target image, ensuring that its content is prioritized during processing.

If, on the other hand, we want to remove an object so that it does not appear in the target image, we do the opposite: we assign the patch a very low weight, effectively close to zero.